Orientation-Aware Seam Carving for Image Retargeting

Arsh Jaswal, 230205 Harshvardhan Gour, 230467
Nikhil Verma, 230699 Shreyas Gupta, 230991

Abstract

Image retargeting must change one color image to a requested height and width. Scaling and cropping ignore what is in the picture, and a seam that only scores the pixels it deletes can cut a smooth subject. We study two variants of forward-energy seam carving. Ours (spectral) adds spectral-residual saliency and an orientation-weighted line map, and it uses no pretrained model. Ours (learned) keeps that seam cost and replaces the saliency map with a pretrained U^2-Net-small model. The seam direction decides which lines a seam is penalized for crossing, inside one forward-energy recurrence. An independent LSD check shows that the in-cost line score overstates the line term's benefit. A cross-evaluation compares the spectral residual with a pretrained saliency map. These comparisons are not a claim of superiority over other published retargeting methods. On eight RetargetMe images the saliency term in Ours (spectral) lowers deleted spectral mass from 0.089 to 0.068, and on a seeded sample of 30 it lowers it from 0.118 to 0.093. The line term lowers crossings of the Hough map it is built from (1.14 to 0.31), but on an independent LSD map it gives no detectable benefit (lower on 16 of 29 images, $p = 0.71$). Both scores reuse maps from the cost, so they show the weights move the seams, not that the pictures look better. Each source scores better under its own map. By eye, the learned map kept the heart on Sanfrancisco, our failure case, but protected the car door instead of the man on obama, and it squeezed the background. A hybrid schedule that stops seams at 75% and scales the rest is judged by eye only.

1. Introduction

Photographs are displayed at aspect ratios their cameras did not use: a phone story, a banner, a slide, a print. The operation that produces the new size is image retargeting. Its input is one color image, a target height, and a target width. Its output is an image of that exact size. The constraint in this project is that the computation stays in the spatial domain. There is no trained retargeting network. Ours (spectral) and Ours (learned) are the two variants tested below. Ours (spectral) uses no pretrained model. Ours (learned) replaces the saliency map with one. The dynamic program runs on images whose long side is at most 640 pixels so that a demo can recompute a result on request.

Two standard operators ignore content. Scaling moves every pixel by the same factor, so a face becomes thinner when the frame becomes narrower. Cropping keeps a window and discards the rest, so an off-center subject can disappear even though the remaining pixels look sharp. Seam carving [1] takes a third route. A vertical seam is an 8-connected path of one pixel per row. Deleting the cheapest such paths changes the width without a global resample. The original cost is a backward energy: the gradient magnitude of the pixels that are removed. That cost does not look at the tear those pixels leave behind, and a smooth subject can be cheaper to cut than a textured background.

Forward energy [4] fixes the tear by scoring the edges that appear after the deletion. It still has no notion of a person or of a window mullion. The saliency term in Ours (spectral) raises the cost of pixels that the spectral residual marks. Those pixels are often edges, not a detected person. Ours (learned) uses a pretrained salient-object map in that same place. A line map raises the cost of long straight segments. Both ideas exist in the literature as separate add-ons. Ours (spectral) puts them in one recurrence, and it does not treat every line the same way. A vertical seam is allowed to cross a horizon, because every vertical seam must, and it is penalized for crossing a column or a mullion. A horizontal seam swaps that preference. Section 3 defines the cost. Section 4 compares it with scaling, cropping, and classic seams, and turns each extra term off in an ablation.

Contributions. The seam direction decides which lines a seam is penalized for crossing, inside one forward-energy recurrence. An independent LSD check

shows that the in-cost line score overstates the line term's benefit. A cross-evaluation compares the spectral map with a pretrained saliency map. The comparison with published RetargetMe operators and a user study are not included.

2. Related work

Avidan and Shamir [1] select seams by dynamic programming on an energy map, remove them to shrink, and duplicate them to enlarge. The energy they use is a function of the image gradient. Rubinstein, Shamir, and Avidan [4] replace that energy with a forward cost that depends on which neighbor the seam arrives from, because the pixels that become adjacent are not known from the gradient alone. The same authors later choose among scaling, cropping, and seam carving with a multi-operator search [5]. We do not search over operators. Scaling and cropping are baselines, and the seam operator is fixed.

Hou and Zhang [2] define a saliency map from the spectral residual of the Fourier amplitude. The residual is the log amplitude minus a local average of the log amplitude, combined with the original phase. The map needs no training images. RetargetMe [6] is the public benchmark used below. It compares several retargeting operators with a user study. We report automatic scores and side-by-side figures on a subset of its source images, and we do not repeat that user study.

3. Method

Figure 1 is the pipeline. Luminance is computed once per seam from the current color image. Saliency and the two line maps are computed once on the image at the start of a shrinking or enlarging stage. When a seam is deleted, the same pixel indices are deleted from those maps. Forward energy is not stored. It is recomputed from the current luminance inside the dynamic program, because it depends on the neighbors that still exist.

3.1. Saliency and line protection

Luminance is the Rec. 601 combination $Y = 0.299R + 0.587G + 0.114B$, scaled to $[0, 1]$. Spectral residual saliency follows Hou and Zhang. The log amplitude of the Fourier transform is subtracted from its 3×3 local average, the original phase is kept, and the inverse transform is squared, blurred with a Gaussian of standard deviation 2.5, and normalized to $[0, 1]$. As an optional comparison, the saliency map S can be replaced by a pretrained U^2-Net-small salient-object model [3], loaded through rembg, and min-max normalized to $[0, 1]$. Its output is almost binary, so it acts as a mask. We use $w_s = 0.25$ for this source, the smallest of five weights tried on Johanneskirche and obama.

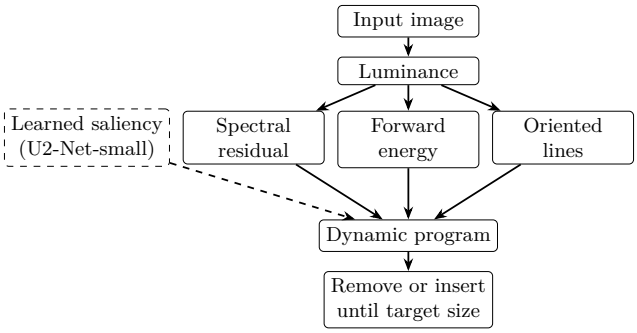

Figure 1. Overview. Forward energy is evaluated on the current luminance. Saliency and the line maps travel with the pixels as seams are removed. Learned saliency is optional and enters the same cost as the spectral residual.

Line segments come from a blurred Canny edge map and the probabilistic Hough transform. The blur (a 5×5 Gaussian with $\sigma = 1.2$) and the Canny thresholds (80 and 160) are there so that brick texture does not fill the mask. A segment is kept only when it is at least 30 pixels long and at least 12% of the short side. At most 40 of the longest segments are drawn, with thickness 2. Let θ be the segment angle from the x-axis. For a vertical seam, which changes the width, the segment is drawn with weight $|\sin\theta|$. A vertical mullion then has weight 1, and a horizontal horizon has weight 0. For a horizontal seam the weight is $|\cos\theta|$. Overlapping draws keep the per-pixel maximum. The horizon still has to be crossed by every vertical seam. The weight stops that forced crossing from also penalizing columns and poles. An edge-density term was tried informally as a way to protect curves and then left out of the cost. It was not measured, and there is no figure for it. A vertical seam crosses a curved outline in a few rows and then travels through the smooth interior, so raising the outline can move the seam into that interior. Figure 2 shows the saliency map, the line map, and the next seam on Johanneskirche. The Hough line map in the cost is unchanged. OpenCV's line-segment detector is not substituted into that cost. Section 4 still scores the Hough map, and it also counts the same seams on a separate LSD map that does not choose them.

Figure 2. Maps on Johanneskirche, the line example. Left to right: the working image, spectral-residual saliency, the line map used by vertical seams (bright yellow marks near-vertical segments), and the next seam chosen by Ours (spectral).

3.2. Dynamic program

Index rows by i and columns by j, and write I for the current luminance. The forward costs of arriving at (i, j) from the three pixels in the row above are

$$C_U(i,j) = |I(i,j{+}1) - I(i,j{-}1)|, \tag{1}$$
$$C_L(i,j) = C_U(i,j) + |I(i{-}1,j) - I(i,j{-}1)|, \tag{2}$$
$$C_R(i,j) = C_U(i,j) + |I(i{-}1,j) - I(i,j{+}1)|. \tag{3}$$

C_U is the new edge between the left and right neighbors that the deletion joins. C_L and C_R add the edge against the pixel the seam just left when the step is diagonal. Out-of-range neighbors are clamped to the border pixel. A step that would leave the image is given infinite cost. The direction-independent term is the saliency S and the line map L_d for the seam direction d,

$$E(i,j) = w_s S(i,j) + w_l L_d(i,j). \tag{4}$$

The cumulative cost and the chosen step are

$$M(i,j) = E(i,j) + \min\big\{M(i{-}1,j{-}1) + C_L,\; M(i{-}1,j) + C_U,\; M(i{-}1,j{+}1) + C_R\big\}. \tag{5}$$

The first row is $M(0,j) = E(0,j)$. Backtracking from the minimum of the last row yields one column index per row. Consecutive indices differ by at most one, so the seam is 8-connected. The spectral weights used for the reported spectral numbers are $w_s = 0.5$ and $w_l = 0.8$. They were chosen by looking at Johanneskirche and obama, which are two of the eight evaluation images, and then frozen. The spectral weights were not swept. The demo sliders can still override them. Section 4 also reports the means with those two images removed, as a sensitivity check rather than a held-out tuning.

Classic seam carving uses the same backtracking with a different recurrence. Its pixel cost is the sum of absolute Sobel responses, and it has no forward term and no saliency or line term. Horizontal seams are vertical seams on the transposed image, with the matching line map transposed as well. While both the height and the width are still above the target, the algorithm computes one seam in each direction and deletes the cheaper one. Enlargement records the seams that would be removed, maps their indices back onto the image before insertion, and writes an averaged copy of each seam pixel next to its right neighbor. One recording pass inserts at most $W-2$ seams, which can almost double the width. The $1.5\times$ enlargement in the experiments fits in that single pass. A second pass runs only when the target is wider than $2W - 2$.

3.3. Hybrid schedule

The hybrid operator keeps Equation (5) and the same weights, and changes only when scaling takes over. If a target side is at least 75% of the original, hybrid is the full model and does not scale. If the target is smaller, seams stop at round($0.75 \times$ original) and cv2.resize finishes that side. Enlargement is unchanged. This is a fixed schedule, not a search over operators [5], and it is judged by eye only.

4. Experiments

4.1. Data and implementation

The images are eight source photographs from RetargetMe [6]: obama and child (people), brick_house and Johanneskirche (buildings), eagle and butterfly (compact objects), waterfall (a landscape), and Sanfrancisco (a busy outdoor scene). The download script keeps these eight files from the public source-image archive and does not redistribute the other 72. Before any operator runs, the long side is limited to 640 pixels with area interpolation. The code is Python 3 with NumPy 2.5.4 and OpenCV 5.0.0. The optional learned comparison uses rembg 2.0.86, onnxruntime 1.29.0, and the u2netp checkpoint (4574861 bytes). No GPU is used. The row update of the dynamic program is vectorized across the width.

The spectral comparison uses five operators. Scale is cv2.resize to the exact target, with area interpolation when both sides shrink. Crop cuts the centered

window of the target aspect ratio and then resizes it to the exact target. Classic is backward-energy seam carving. Ours (spectral) is Equation (5) with $w_s = 0.5$ and $w_l = 0.8$. Ours (learned) is that same recurrence with the pretrained map, $w_s = 0.25$ and $w_l = 0.8$, and Section 4.3 reports it. Hybrid is that same cost, with seams stopped at 75% of the original side and a uniform scale after that. At a 75% target the two are the same operator. Hybrid is reported on the 50% width cut. The ablation uses the same recurrence with (w_s, w_l) set to $(0, 0)$ (forward energy alone), $(0.5, 0)$ (saliency only), and $(0, 0.8)$ (lines only).

The width is reduced to 75% and to 50% of the working width, with the height held fixed. The 75% setting is a 25% reduction and is the one used for the ablation and the runtime. waterfall is also reduced to 75% height, and butterfly is enlarged to 150% width. Every reported output was checked to have the requested height and width.

Three seam scores are reported. The saliency fraction is the sum of the spectral-residual map on pixels that are actually deleted, divided by the sum of that map. Insertion does not delete pixels, so the fraction is not used for the enlargement. The Hough line score is the number of seam pixels that land on a positive value of the Hough map, per 100 seam pixels. The LSD line score is the same count on an LSD map. The saliency fraction and the Hough score use maps that the full model puts in its cost. The LSD map does not. Classic seams are scored with those maps even though their recurrence does not use them. A lower number therefore shows that the weight changed which pixels were deleted. It does not, by itself, show that a viewer would prefer the result, and it does not measure tears in regions the maps do not mark. Scale and crop do not choose seams, so they have no entry on these three scores. Runtime is one wall-clock measurement of the whole call. Before the 30-image sample, the original eight were carved again at 75% width. The saliency fraction and the Hough score matched metrics.csv on every seam method, including brick_house classic at 0.153154 and 3.2319. That sample is 30 source images drawn with seed 604. Three of those 30 are also in the original eight: child, Sanfrancisco, and waterfall. LSD is counted in the same pass that chooses the seams. The run writes a CSV only. Contact sheets stay the original eight.

4.2. Results

Table 1 gives the mean and sample standard deviation at 75% width. Those standard deviations, Table 3, and the six-image check later in this section are the printout of experiments/report_numbers.py on the saved metrics.csv. They are the worked example. Table 5 and its sign tests are reported with the learned comparison in Section 4.3. They are the printout of experiments/run_n30.py, saved as n30_summary.txt. Table 2 is the 50% width cut. Table 3 is the per-image breakdown at 75% width. Figure 3 shows four of the eight images at 75% width: the church (lines), the portrait (saliency), the street sculpture (a busy scene), and the waterfall. Figure 4 repeats that set at 50% width. Figure 5 isolates the terms on the church. Figure 6 is the 50% cut of Sanfrancisco. Figure 8 and Figure 9 are the height reduction and the enlargement.

Table 1. Worked example, the original eight images, at 75% width. Entries are mean (sample standard deviation). Saliency is the fraction of spectral-residual mass deleted. Lines are pixels of the Hough map per 100 seam pixels. Seconds are one run per image. This is not the 30-image sample.

Method	Saliency	Lines	Seconds
Scale	–	–	0.00
Crop	–	–	0.00
Classic	0.089 (0.045)	1.30 (1.25)	11.0 (1.5)
Forward only	0.089 (0.039)	1.14 (1.18)	13.7 (1.4)
Forward + saliency	0.068 (0.035)	0.99 (1.09)	12.9 (3.0)
Forward + lines	0.089 (0.040)	0.31 (0.22)	13.4 (3.3)
Ours (spectral)	0.068 (0.036)	0.37 (0.31)	14.4 (1.7)

Table 2. Worked example, the same original eight images, at 50% width. Mean (sample standard deviation). Not the 30-image sample.

Method	Saliency	Lines	Seconds
Scale	–	–	0.00
Crop	–	–	0.00
Classic	0.222 (0.097)	1.61 (1.34)	21.5 (3.9)
Ours (spectral)	0.186 (0.090)	0.69 (0.56)	26.0 (3.3)

At a 25% width reduction the two extra terms move different columns of Table 1, and the standard deviations are large. Forward energy alone and classic backward energy both have mean saliency fraction 0.089. Adding saliency lowers that mean to 0.068. Adding the line map without saliency leaves the saliency mean at 0.089 and lowers the line score from 1.14 to 0.31. Ours (spectral) matches the saliency-only mean, 0.068, so on this score the line weight does not undo the saliency weight. On the line score Ours (spectral) is 0.37, which is still far below forward energy and a little worse than lines alone. Table 3 is that comparison image by image. Means that print as the same three decimals are not the same seams. At 50% width the means move the same way, 0.186 against 0.222 for saliency and 0.69 against 1.61 for lines. Table 4 is that cut image by image.

On the original eight, scored as a worked example, an LSD line map that is not in the cost gives Ours

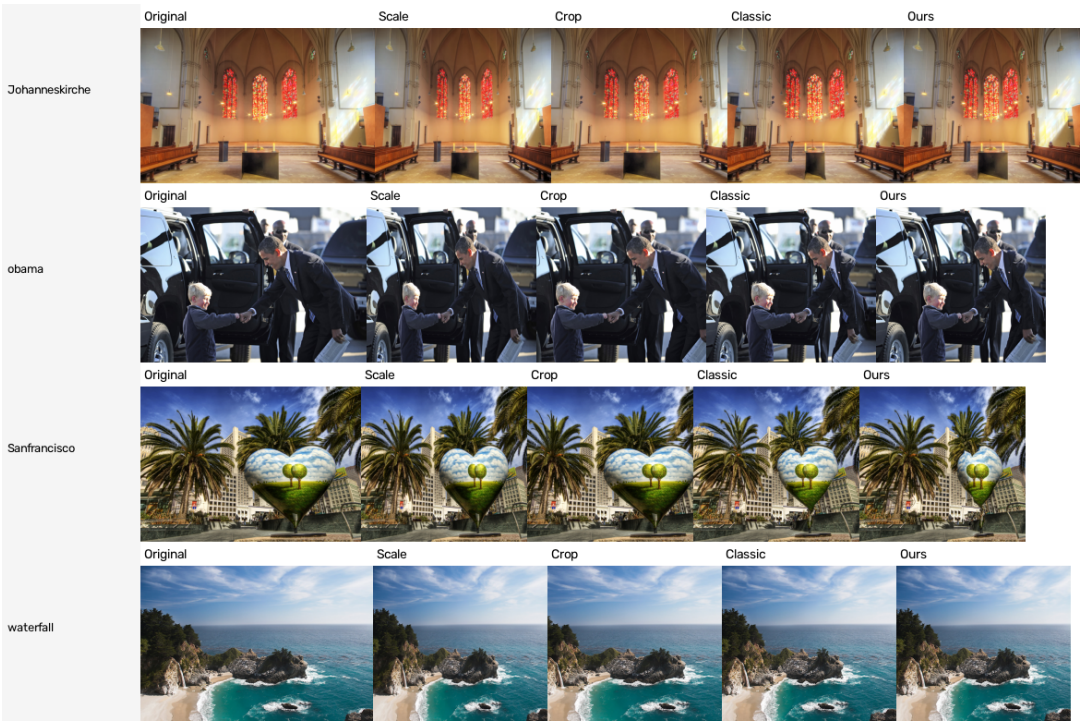

Figure 3. Width reduced to 75%. Columns are the original, uniform scale, center crop, classic seams, and Ours (spectral).

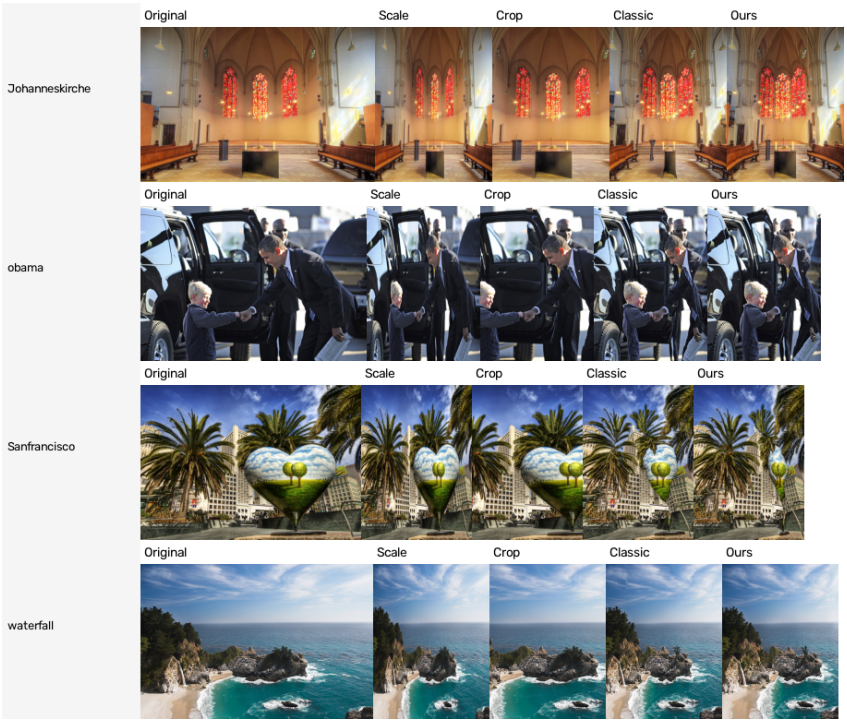

Figure 4. Width reduced to 50% on the same four images. Any impression that one seam method bends a window or an outline less than the other is qualitative. The measured claims are the scores in Tables 1–3.

Figure 5. Ablation on Johanneskirche at 75% width. Forward energy alone, forward energy plus saliency, forward energy plus lines, Ours (spectral), and classic backward energy.

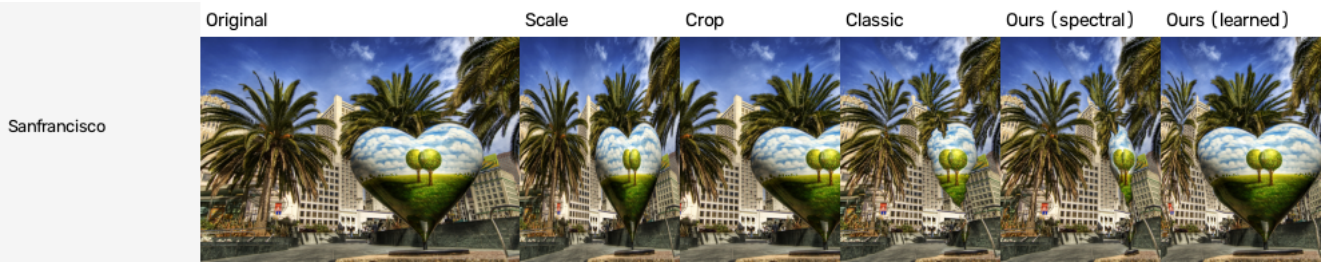

Figure 6. Sanfrancisco at 50% width. The crop cuts the heart, scaling squeezes it, and both spectral-based seam methods pinch it. Ours (learned), in the last column, keeps the heart at the cost of the background. This is a judgment by eye on one image.

(spectral) 1.26 (0.54) crossings per 100 seam pixels against 1.45 (0.73) for classic at 75% width, a 13% drop versus 72% on the Hough map, and it is lower on only four of eight images. At the same width, forward energy alone is already 1.28 (0.55) and adding the line map leaves the score at 1.27 (0.53). At 50% width Ours (spectral) is higher on Sanfrancisco and waterfall and lower on the other six, including brick_house and obama by 0.002 and 0.007. The line term therefore mostly steers seams away from Hough-detected lines, and this eight-image check does not show that it protects lines in general or that the picture looks better. The weights were chosen on Johanneskirche and obama. Dropping those two and averaging the other six, at 75% width, leaves the same separation: classic 0.096 and 1.45, forward 0.095 and 1.19, saliency only 0.074 and 1.01, lines only 0.095 and 0.28, Ours (spectral) 0.075 and 0.33, again as saliency fraction and line crossings. This only checks that the direction is not confined to the two inspected images. It is not a retune on held-out data.

Figures 3 and 4 are qualitative. At 75% width, classic seams and Ours (spectral) look close to each other, while scaling compresses the frame and the center crop cuts the heart in Sanfrancisco and clips the child in obama. At 50% width the sheets suggest less bending in the church window and in the child's outline under Ours (spectral). Those sheets are small, and this paper does not treat that impression as a measurement. Figure 6 puts Ours (learned) beside that failure. The crop cuts the heart, scaling squeezes it, and both spectral-based seam methods pinch it. The last column keeps the heart at the cost of the background. That is a judgment by eye on one image. obama also degrades under every method at 50% width, because that scene is packed with content. Figure 7 is the same 50% width with the hybrid schedule added. Figure 14 is the full eight-image hybrid sheet. The hybrid method is judged by eye only, with no scores. The hybrid column is the 75% seam result scaled down to half width. It does not receive the extra seams that, under Ours (spectral), continue from 75% to 50% and cut the heart further. It is also not the scaled original: the first 25% of seams have already narrowed the sculpture relative to uniform scaling. The saliency and line scores are not quoted for hybrid. Those scores count only the seams, and hybrid deletes half as many seams before the scale, so a lower fraction would not mean a better picture. The waterfall height cut and the butterfly enlargement are single images. On the height cut the horizontal line score is 0 for both seam methods, so that map did nothing there; the saliency fractions are 0.009 and 0.016, which is one pair of numbers, not a mean. The enlargement reaches 150% width. Scaling stretches the butterfly. Seam insertion spends the extra width on the background by writing averaged copies of the recorded seams.

A 25% width cut takes about 11 s for classic seams and about 14 s for the forward variants. Scaling and

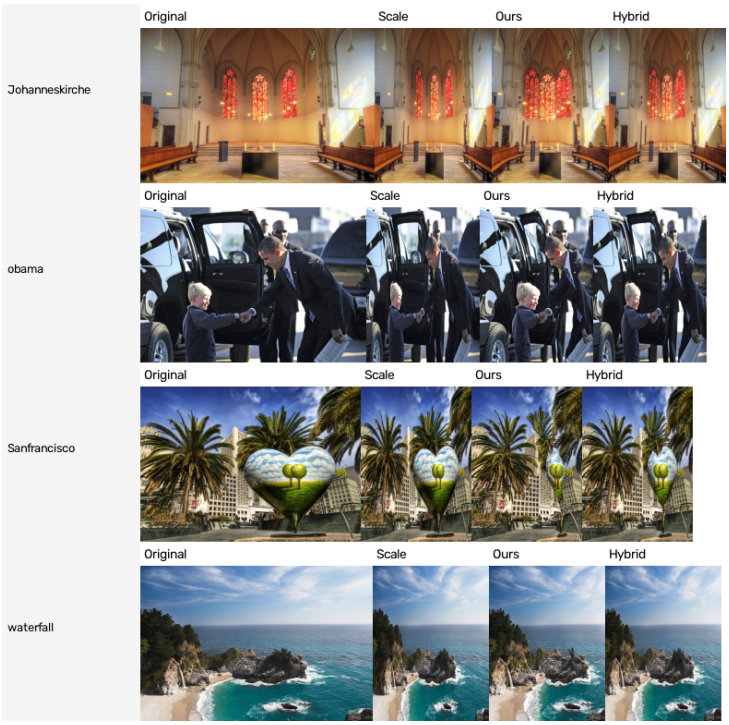

Figure 7. Width reduced to 50%. Columns are the original, uniform scale, Ours (spectral) carved all the way, and hybrid (seams until 75% of the original width, then scale). Hybrid avoids only the damage that the extra seams would add past 75%.

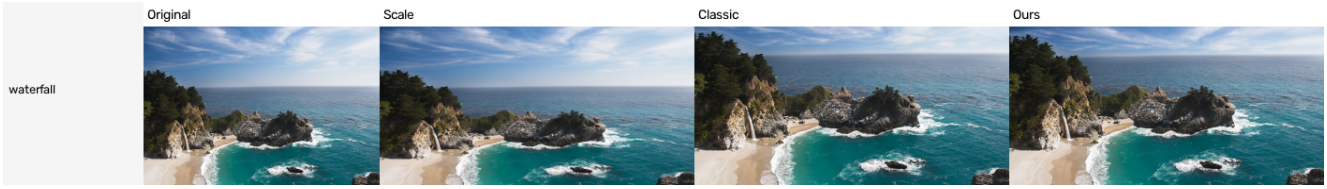

Figure 8. Height reduced to 75% on waterfall. Horizontal seams use the line map weighted by $|\cos\theta|$.

cropping finish in well under 0.01 s. The maps are computed once per call, so they do not explain the differences among the forward rows in Table 1. Those differences sit inside the standard deviation across the eight images. Each image was timed once, so that spread is not a repeat of the same image. The steadier gap is between classic seams and any forward recurrence, which is recomputed on every seam.

The full eight-image contact sheets are Figures 12 and 13 in the appendix. Across the 75% and 50% trials, every output size matched the requested size.

4.3. Learned saliency

Ours (spectral) needs no pretrained model. Ours (learned) replaces only the saliency map. On the seeded sample of 30 the line term does not show an independent benefit, and that is the setting in which the two variants are compared.

Table 5 is that sample for Ours (spectral). Ours (spectral) is lower than classic on the LSD score for 12 images, higher for 17, and tied for one. A two-sided exact sign test on the 29 non-ties gives $p = 0.46$. Adding the line weight to forward energy is lower on 16 images, higher on 13, and tied on one ($p = 0.71$). The eight-image LSD drop does not recur. The mean for the line weight is lower than forward energy alone, 1.16 against 1.35, but the per-image signs do not support that as a general benefit, and both standard deviations are about as large as the means. On the Hough map, which is still inside the cost, the same 30 images still move: classic 1.70 (1.45) against Ours (spectral) 1.03 (0.86), and forward energy 2.07 (1.75) against 0.94 (0.75) with the line weight. That Hough drop is the circular score. On spectral-residual saliency the reduction

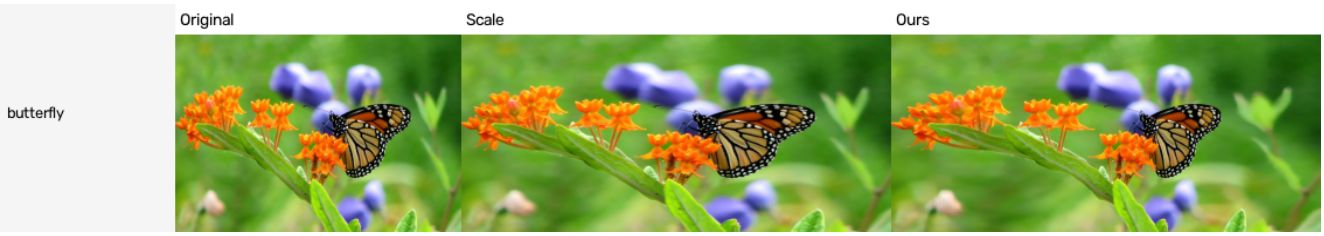

Figure 9. Width enlarged to 150% on butterfly. Seams are recorded by removal and inserted as averaged duplicates.

Table 3. Worked example: per-image Hough scores on the original eight at 75% width, written as saliency fraction / line crossings per 100 seam pixels. Means that agree to three decimals in Table 1 are not the same seams. Images marked † are the two used to choose w_s and w_l, and they are left out of the 30-image headline.

Image	Classic	Forward	+Saliency	+Lines	Ours (spectral)
brick_house	0.153/3.23	0.142/1.82	0.120/1.62	0.142/0.33	0.124/0.46
butterfly	0.043/0.16	0.055/0.36	0.037/0.22	0.052/0.04	0.036/0.07
child	0.023/0.00	0.033/0.13	0.020/0.03	0.032/0.07	0.020/0.01
eagle	0.108/1.62	0.094/0.47	0.075/0.28	0.090/0.13	0.074/0.13
Johanneskirche†	0.070/0.94	0.071/0.97	0.055/0.90	0.071/0.35	0.055/0.38
obama†	0.069/0.74	0.066/1.02	0.042/0.97	0.067/0.47	0.042/0.57
Sanfrancisco	0.137/3.09	0.134/3.76	0.108/3.37	0.139/0.71	0.109/0.95
waterfall	0.110/0.60	0.114/0.59	0.086/0.52	0.114/0.38	0.087/0.37

holds. Classic is 0.118 (0.037) and Ours (spectral) is 0.093 (0.031), lower on all 30 images (two-sided sign test $p = 1.86 \times 10^{-9}$). Forward energy alone is 0.121 (0.035), and forward plus the saliency weight is 0.091 (0.032).

The learned map is U^2-Net-small, loaded through rembg as the u2netp checkpoint [3]. It is pretrained. We did not train it. Its output is near-binary, so in the seam cost it acts like a mask. The spectral default remains $w_s = 0.5$ and $w_l = 0.8$. For Ours (learned), $w_l = 0.8$ and $w_s = 0.25$. That 0.25 was the smallest of the five weights $\{0.25, 0.5, 1, 2, 4\}$ tested on Johanneskirche and obama only. Every tested weight put the learned-mass fraction below half the forward-only value on both images, and the fraction of seam pixels above 0.5 on the learned map was 0 at every tested weight. obama is one of the two tuning images, and its learned map marks the car door rather than the people, so this weight is not evidence that people are protected. Ours (learned) always includes the line weight. It is compared with forward plus lines and with Ours (spectral), which also use $w_l = 0.8$. It is not compared with forward-only, because that comparison confounds the line weight. Ours (learned) uses $w_s = 0.25$ and Ours (spectral) uses $w_s = 0.5$, so the comparison is also not a pure swap of the map. Each source is expected to score better under its own map. On the 30-image sample, Ours (spectral) scores 0.093 (0.031) under the spectral map and 0.179 (0.138) under the learned map. Ours (learned) scores 0.144 (0.052) under the spectral map and 0.030 (0.053) under the learned map. Against Ours (spectral) under the spectral map, Ours (learned) is lower on 0 images, higher on 30, and tied on 0 ($p = 1.86 \times 10^{-9}$). This does not show that learned-chosen seams are better on this map. Against Ours (spectral) under the learned map, Ours (learned) is lower on 30 images, higher on 0, and tied on 0 ($p = 1.86 \times 10^{-9}$). The sign test favors Ours (learned) on this map. Against forward + lines under the spectral map, Ours (learned) is lower on 11 images, higher on 19, and tied on 0 ($p = 0.2005$). This does not show that learned-chosen seams are better on this map. Against forward + lines under the learned map, Ours (learned) is lower on 30 images, higher on 0, and tied on 0 ($p = 1.86 \times 10^{-9}$). The sign test favors Ours (learned) on this map. Ours (learned) helps when the marked object is compact: by eye it keeps the heart in Figure 6, which Ours (spectral) pinches. It hurts when the map marks the wrong region: on obama it protects the car door rather than the man, and the background is squeezed. The line-protection cost is in Tables 6 and 8. At 50% width the Hough score is 0.69 for Ours (spectral) and 1.58 for Ours (learned). Tables 6 and 7 are the 75% width cross-evaluation. Table 8 is the same six methods on the original eight at 50% width. The earlier full sheets are Figures 12 and 13. The same eight images with Ours (spectral) and Ours (learned) are Figures 16 and 15. The 30-image sample is Figures 17 and 18, in alphabetical order. Those sheets are impressions, not measurements, and the images were not selected by outcome. Figure 11 is Sanfrancisco at 50% width, and Figure 10 is brick_house, child, and obama

Table 4. Worked example: the original eight at 50% width. Each cell is saliency fraction / Hough crossings / LSD crossings per 100 seam pixels, for classic seams and Ours (spectral). These cells are the printout of experiments/report_numbers.py. This is not Table 5. Images marked † were used to choose the weights.

Image	Classic	Ours (spectral)
brick_house	0.349/3.61/1.46	0.307/0.95/1.46
butterfly	0.131/0.48/0.66	0.105/0.17/0.53
child	0.090/0.45/2.49	0.069/0.25/2.01
eagle	0.269/1.27/1.77	0.218/0.31/0.87
Johanneskirche†	0.159/1.10/1.64	0.133/0.58/1.60
obama†	0.173/1.10/1.77	0.129/0.80/1.76
Sanfrancisco	0.340/3.85/1.23	0.305/1.91/1.32
waterfall	0.260/1.04/0.72	0.218/0.51/0.80

Table 5. Seeded sample of 30 RetargetMe images at 75% width, seed 604. Johanneskirche and obama were not drawn, so this headline is all 30 images and is not Table 1. Entries are mean (sample standard deviation). Saliency is the fraction of spectral-residual mass deleted. Hough and LSD are crossings per 100 seam pixels. The Hough column reuses the map in the cost. The LSD column does not.

Method	Saliency	Hough	LSD
Classic	0.118 (0.037)	1.70 (1.45)	1.08 (0.93)
Forward only	0.121 (0.035)	2.07 (1.75)	1.35 (1.37)
Forward + saliency	0.091 (0.032)	1.87 (1.75)	1.42 (1.51)
Forward + lines	0.122 (0.036)	0.94 (0.75)	1.16 (0.94)
Ours (spectral)	0.093 (0.031)	1.03 (0.86)	1.26 (1.10)

at the same width. Both are impressions, not measurements. Blob is the fraction of seam pixels whose learned-map value is above 0.5.

Table 6. Original eight at 75% width. Spectral and Learned are fractions of that map deleted. Hough and LSD are crossings per 100 seam pixels. Blob is the fraction of seam pixels with learned value above 0.5. Ours (learned) uses $w_s = 0.25$ and $w_l = 0.8$. Ours (spectral) uses $w_s = 0.5$ and $w_l = 0.8$.

Method	Spectral	Learned	Hough	LSD	Blob
Classic	0.089 (0.045)	0.192 (0.155)	1.30 (1.25)	1.45 (0.73)	0.120 (0.135)
Forward only	0.089 (0.039)	0.196 (0.172)	1.14 (1.18)	1.28 (0.55)	0.141 (0.182)
Forward + lines	0.089 (0.040)	0.212 (0.212)	0.31 (0.22)	1.27 (0.53)	0.157 (0.207)
Ours (spectral)	0.068 (0.036)	0.210 (0.225)	0.37 (0.31)	1.26 (0.54)	0.159 (0.219)
Learned saliency only	0.114 (0.081)	0.005 (0.008)	2.16 (3.05)	1.02 (0.65)	0.002 (0.005)
Ours (learned)	0.118 (0.086)	0.005 (0.009)	0.94 (1.46)	0.98 (0.62)	0.002 (0.005)

Table 7. Seeded 30-image sample at 75% width, seed 604. Johanneskirche and obama were not drawn. Columns match Table 6. Ours (learned) is not claimed to be better unless the sign tests say so.

Method	Spectral	Learned	Hough	LSD	Blob
Classic	0.118 (0.037)	0.167 (0.108)	1.70 (1.45)	1.08 (0.93)	0.156 (0.191)
Forward only	0.121 (0.035)	0.184 (0.119)	2.07 (1.75)	1.35 (1.37)	0.174 (0.205)
Forward + lines	0.122 (0.036)	0.183 (0.129)	0.94 (0.75)	1.16 (0.94)	0.171 (0.211)
Ours (spectral)	0.093 (0.031)	0.179 (0.138)	1.03 (0.86)	1.26 (1.10)	0.172 (0.212)
Learned saliency only	0.145 (0.054)	0.029 (0.052)	2.27 (2.03)	1.29 (1.37)	0.052 (0.132)
Ours (learned)	0.144 (0.052)	0.030 (0.053)	1.17 (0.98)	1.10 (0.96)	0.053 (0.137)

Table 8. Original eight at 50% width. Same columns and weights as Table 6.

Method	Spectral	Learned	Hough	LSD	Blob
Classic	0.222 (0.097)	0.377 (0.225)	1.61 (1.34)	1.47 (0.60)	0.127 (0.116)
Forward only	0.231 (0.087)	0.395 (0.240)	1.56 (1.28)	1.33 (0.53)	0.142 (0.138)
Forward + lines	0.224 (0.097)	0.403 (0.301)	0.63 (0.48)	1.28 (0.50)	0.147 (0.156)
Ours (spectral)	0.186 (0.090)	0.400 (0.314)	0.69 (0.56)	1.29 (0.52)	0.147 (0.164)
Learned saliency only	0.288 (0.183)	0.037 (0.046)	2.50 (3.01)	1.25 (0.64)	0.011 (0.018)
Ours (learned)	0.285 (0.179)	0.048 (0.044)	1.58 (1.98)	1.21 (0.60)	0.015 (0.018)

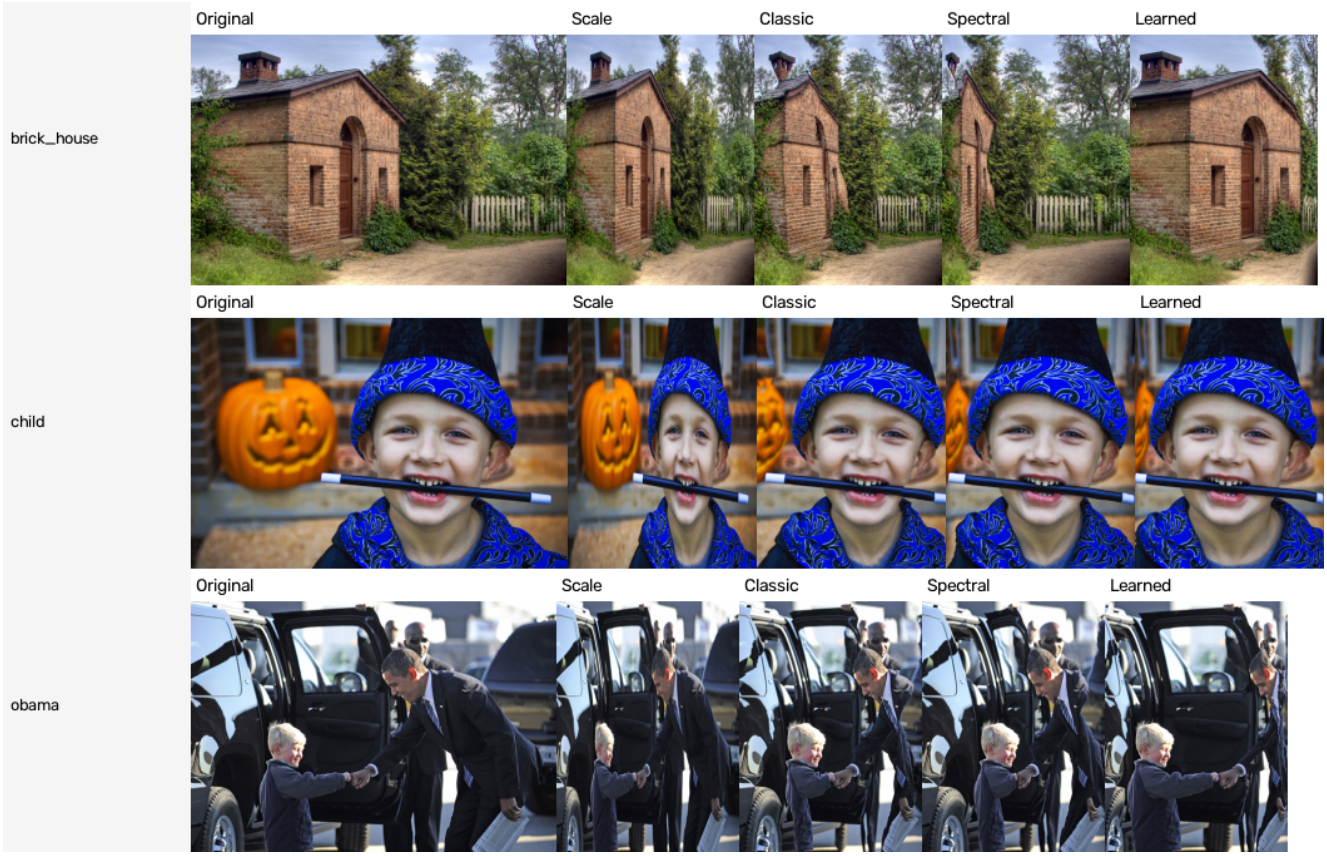

Figure 10. brick_house, child, and obama at 50% width. Columns are the original, uniform scale, classic seams, Ours (spectral), and Ours (learned). These sheets are impressions, not measurements. The obama learned map marks the car door rather than the people.

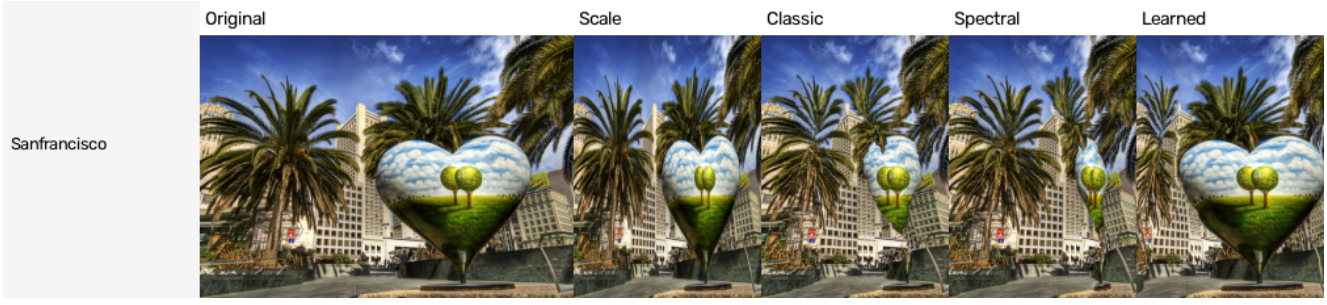

Figure 11. Sanfrancisco at 50% width, with the same columns as Figure 10. Any impression that the heart is preserved is not a measurement.

4.4. What the demo shows

The Gradio application loads an image, caps the long side at 640 pixels, and takes the target size as a percentage of that working image. The user picks scale, crop, classic, Ours (spectral), or the hybrid schedule, and moves w_s and w_l. Optional panels show the saliency map, the vertical-seam line map, and the next vertical seam. Three examples are wired to the roles above: Johanneskirche for lines, obama for a person, and Sanfrancisco for the busy failure. A useful order in a presentation is to raise w_l on the church, compare classic and Ours (spectral) on the portrait, and then set the street scene to 50% width and say plainly what every method loses.

4.5. Limitations

The saliency score and the Hough line score reuse the maps inside the cost. A lower score shows that the corresponding weight changed the seam. It does not show that the deleted pixels were less important to a viewer, and pushing seams off those maps can tear texture that neither map marks. A pretrained saliency model is reported as an optional comparison. No summed forward-energy of the chosen seams is reported. One independent check is reported: the same seams are counted on an LSD line map that is not in the cost. On the original eight that score moves much less than the Hough score. On the seeded 30 it does not favor Ours (spectral) over classic seams, and the line weight does not win a sign test against forward energy. Spectral residual saliency responds strongly to edges and is not a subject detector. The optional learned comparison is a pretrained salient-object model [3]. The line map only contains segments the Hough transform returns, so short and curved structures, including the heart sculpture, are unprotected except where the residual happens to be high. A strong-edge penalty does not fix that curve: the seam pays it at the crossing and then follows the cheap interior. The hybrid schedule stops further seam cuts after 75%, which is not the same as protecting the object during the first 25%. The spectral weights were chosen on two of the eight test images, and they were not swept. Those two were not in the 30-image draw. The line-score standard deviation is about as large as the mean on both the eight and the 30. Thirty images is still not the full RetargetMe set. The comparison does not include the published RetargetMe operators, such as multi-operator search, shift-map, or warping. Hybrid is a fixed seam-then-scale schedule, not that search. The height cut and the enlargement are single images. Runtimes are one measurement each. There is no user study.

5. Conclusion

The operator is forward-energy seam carving plus a spectral-residual term and an orientation-weighted line term. On eight RetargetMe images, turning the terms off one at a time moves two different scores, and both scores are built from the maps in the cost. The saliency weight lowers deleted residual mass from 0.089 to 0.068. On the seeded 30 images Ours (spectral) is lower than classic on that fraction for every image, 0.093 (0.031) against 0.118 (0.037). The line weight lowers Hough-map crossings from 1.14 to 0.31 per 100 seam pixels. Ours (spectral) keeps the saliency-score reduction and most, not all, of that Hough reduction: its Hough score is 0.37 rather than 0.31. Leaving out the two images used to pick the weights does not remove that Hough and saliency separation on the other six. That is evidence that those weights move the maps they were built from. On the original eight, an LSD map that is not in the cost puts Ours (spectral) only about 13% below classic seams, and only on four of the eight images, and adding the line weight to forward energy does not lower that score. On the seeded sample of 30 the LSD mean goes the other way, 1.26 against 1.08, and the sign tests do not show a benefit ($p = 0.46$ for Ours (spectral) against classic, $p = 0.71$ for the line weight against forward energy). The Hough score on those 30 images still falls, which is what a map inside the cost is expected to do. It is not evidence that the results look better, that subjects are preserved, or that the method beats other published retargeting operators. Scaling and cropping are the simple baselines. A large reduction of a busy curved scene still damages the content. Stopping the seams at 75% and scaling the rest removes

only the extra damage that comes from carving on to half width. It does not put the curved sculpture back to the shape that uniform scaling keeps. On the learned comparison, each source wins under its own map. The one fair comparison, Ours (learned) against forward plus lines under the spectral map, is not significant ($p = 0.20$). By eye, the learned map kept the heart on Sanfrancisco, our failure case, but protected the car door instead of the man on obama, and it squeezed the background. Ours (spectral) needs no model but fails on curved objects like the heart. Ours (learned) keeps compact salient objects but protects the largest object, not necessarily the subject, and it costs line protection.

References

[1] Shai Avidan and Ariel Shamir. Seam carving for content-aware image resizing. In ACM SIGGRAPH 2007 Papers, pages 10:1–10:9, 2007. 1, 2

[2] Xiaodi Hou and Liqing Zhang. Saliency detection: A spectral residual approach. In IEEE Conference on Computer Vision and Pattern Recognition, pages 1–8, 2007. 2

[3] Xuebin Qin, Zichen Zhang, Chenyang Huang, Masood Dehghan, Osmar R. Zaiane, and Martin Jagersand. U^2-Net: Going deeper with nested U-structure for salient object detection. Pattern Recognition, 106: 107404, 2020. 2, 8, 12

[4] Michael Rubinstein, Ariel Shamir, and Shai Avidan. Improved seam carving for video retargeting. ACM Transactions on Graphics, 27(3):16:1–16:9, 2008. 1, 2

[5] Michael Rubinstein, Ariel Shamir, and Shai Avidan. Multi-operator media retargeting. ACM Transactions on Graphics, 28(3):23:1–23:11, 2009. 2, 3

[6] Michael Rubinstein, Diego Gutierrez, Olga Sorkine, and Ariel Shamir. A comparative study of image retargeting. ACM Transactions on Graphics, 29(6):160:1–160:11, 2010. 2, 3

A. Individual contributions

A.1. Arsh Jaswal (230205)

I implemented the forward-energy and classic backward-energy dynamic programs, seam removal and insertion, and enlargement in seam.py.

A.2. Harshvardhan Gour (230467)

I implemented spectral-residual saliency and the orientation-weighted Hough line map in energy.py.

A.3. Nikhil Verma (230699)

I implemented the uniform scaling and center-crop baselines, the hybrid schedule that stops seams at 75% of a side, and the RetargetMe experiments.

A.4. Shreyas Gupta (230991)

I implemented the Gradio demo and the optional learned-saliency comparison.

All four of us reviewed the experiments, checked the numbers against report_numbers.py output, and wrote the report together.

B. Links

Code repository: <GITHUB URL>. The project repository is https://github.com/shreyas-gupta-23/ee604-image-retargeting. The source images are the RetargetMe archive of Rubinstein et al.: http://people.csail.mit.edu/mrub/retargetme/ and the file linked as "source images only" from http://people.csail.mit.edu/mrub/retargetme/download.html. The libraries are OpenCV (https://github.com/opencv/opencv), NumPy (https://github.com/numpy/numpy), Gradio (https://github.com/gradio-app/gradio), rembg (https://github.com/danielgatis/rembg), ONNX Runtime (https://github.com/microsoft/onnxruntime), and U^2-Net (https://github.com/xuebinqin/U-2-Net). The u2netp weight file is the checkpoint rembg retrieves from https://github.com/danielgatis/rembg/releases/download/v0.0.0/u2netp.onnx. The source is printed at the end of this appendix.

C. All eight images

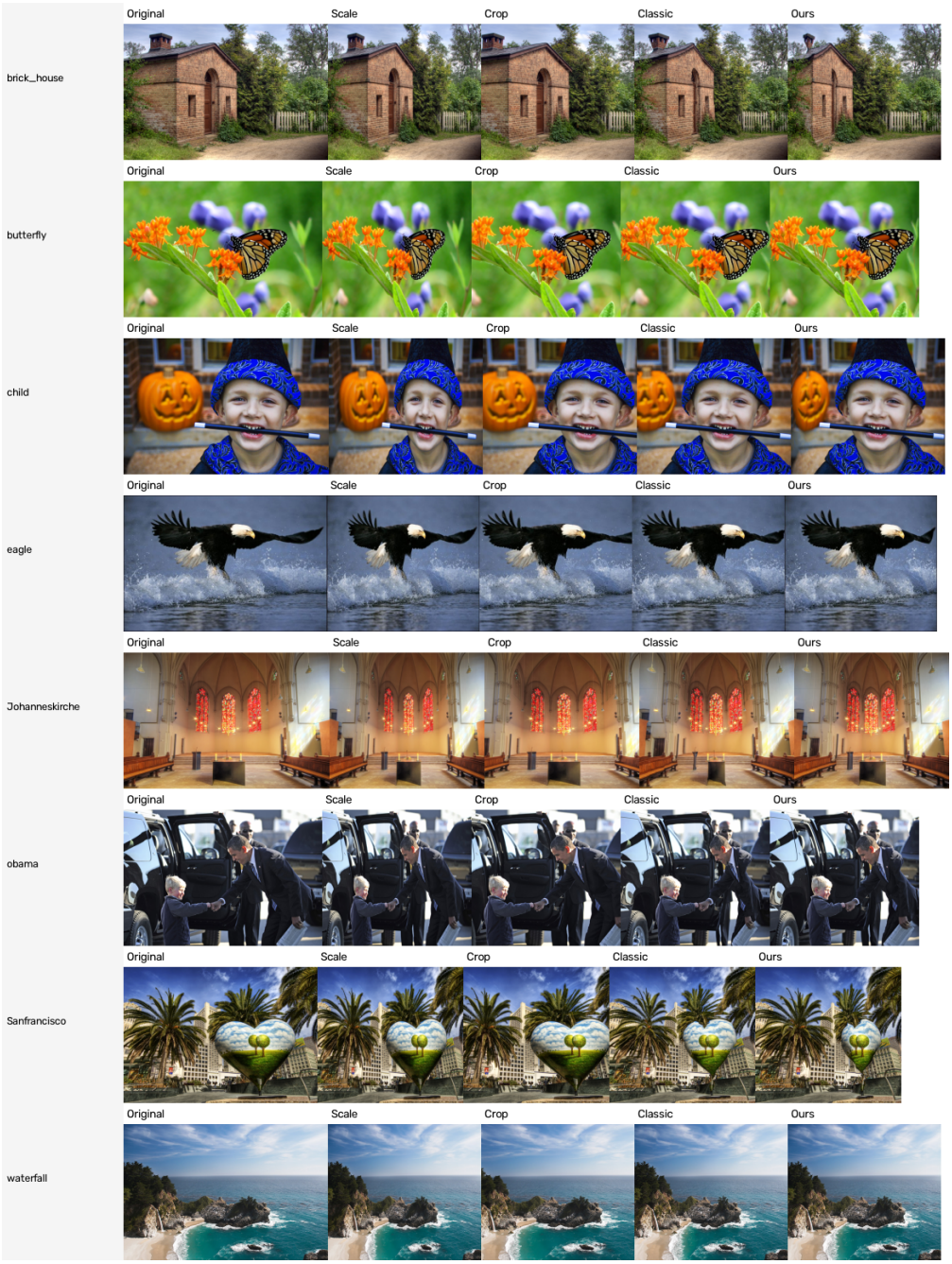

Figure 12. All eight images at 75% width. Columns are the original, scale, crop, classic seams, and Ours (spectral).

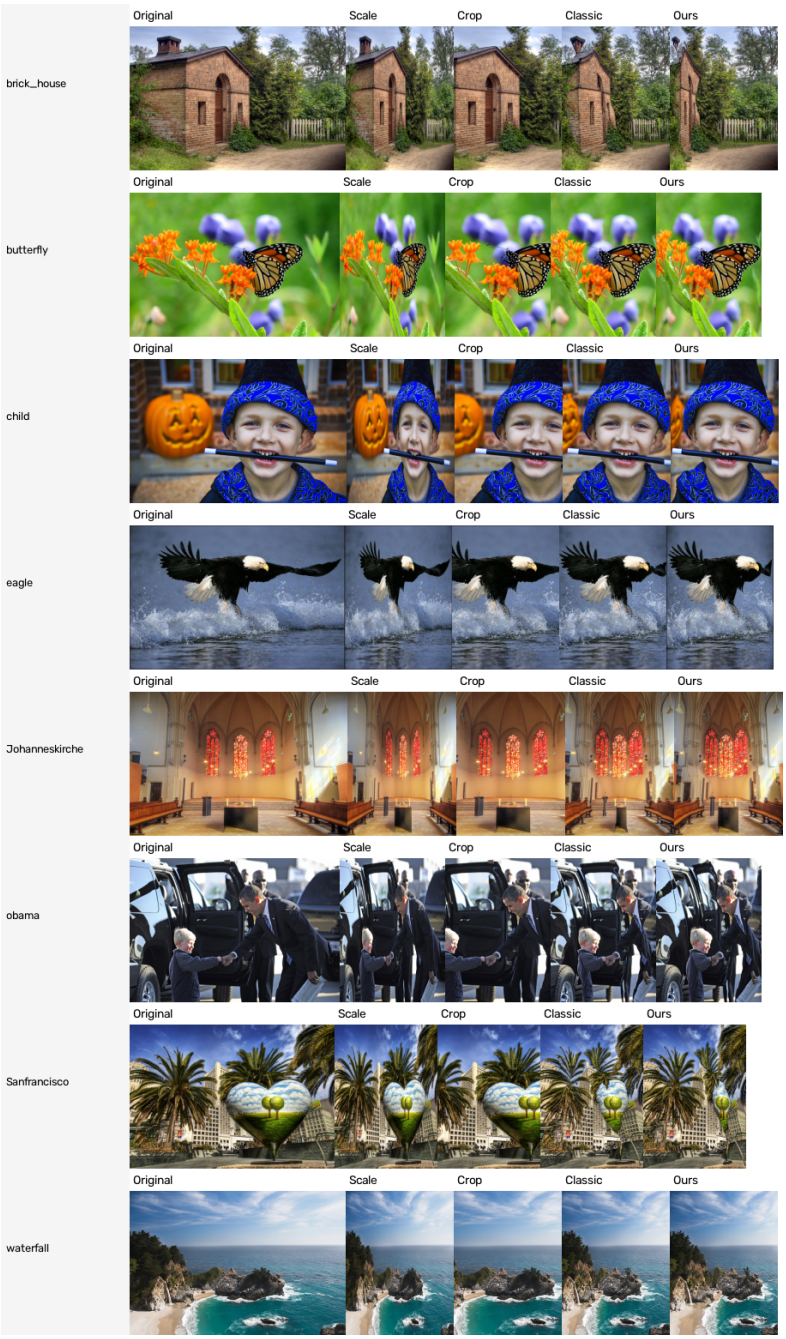

Figure 13. All eight images at 50% width.

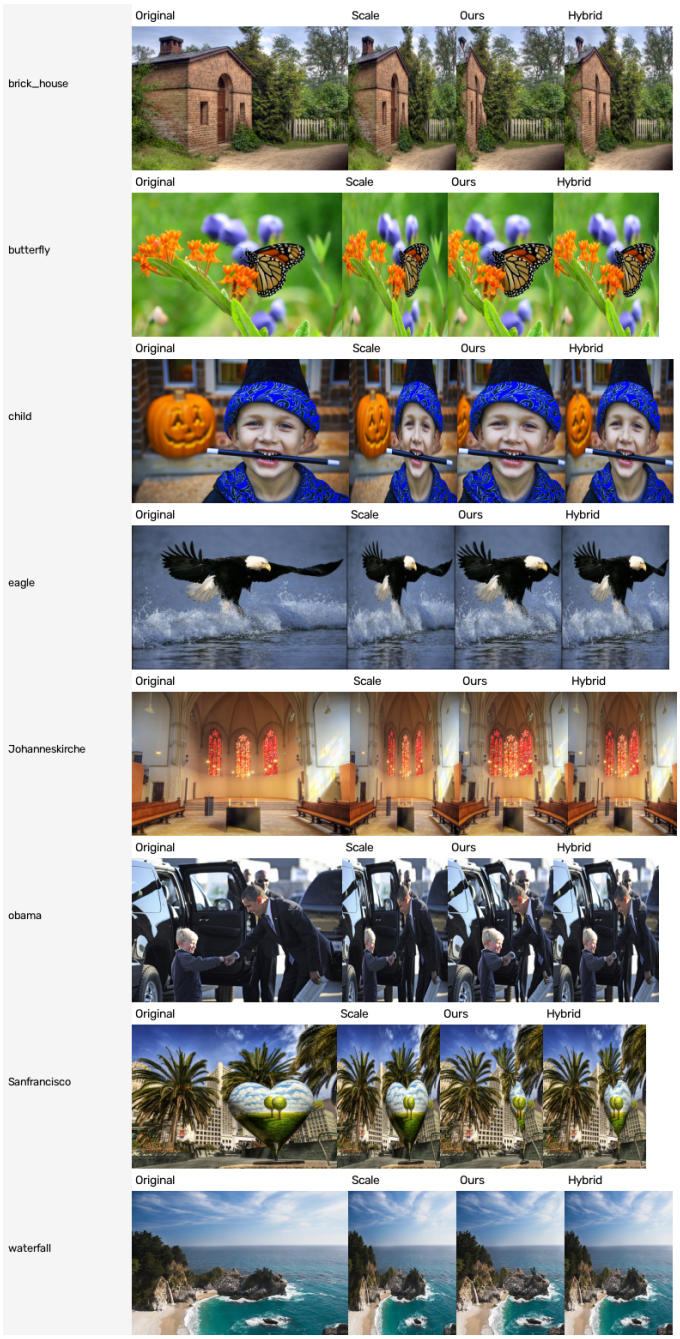

Figure 14. All eight images at 50% width. Columns are the original, uniform scale, Ours (spectral), and hybrid.

D. Learned comparison sheets

Figure 15. All eight original images at 50% width, in the same order as Figures 12 and 13. Columns are the original, uniform scale, classic seams, Ours (spectral), and Ours (learned). These are impressions, not measurements. The images were not selected by outcome.

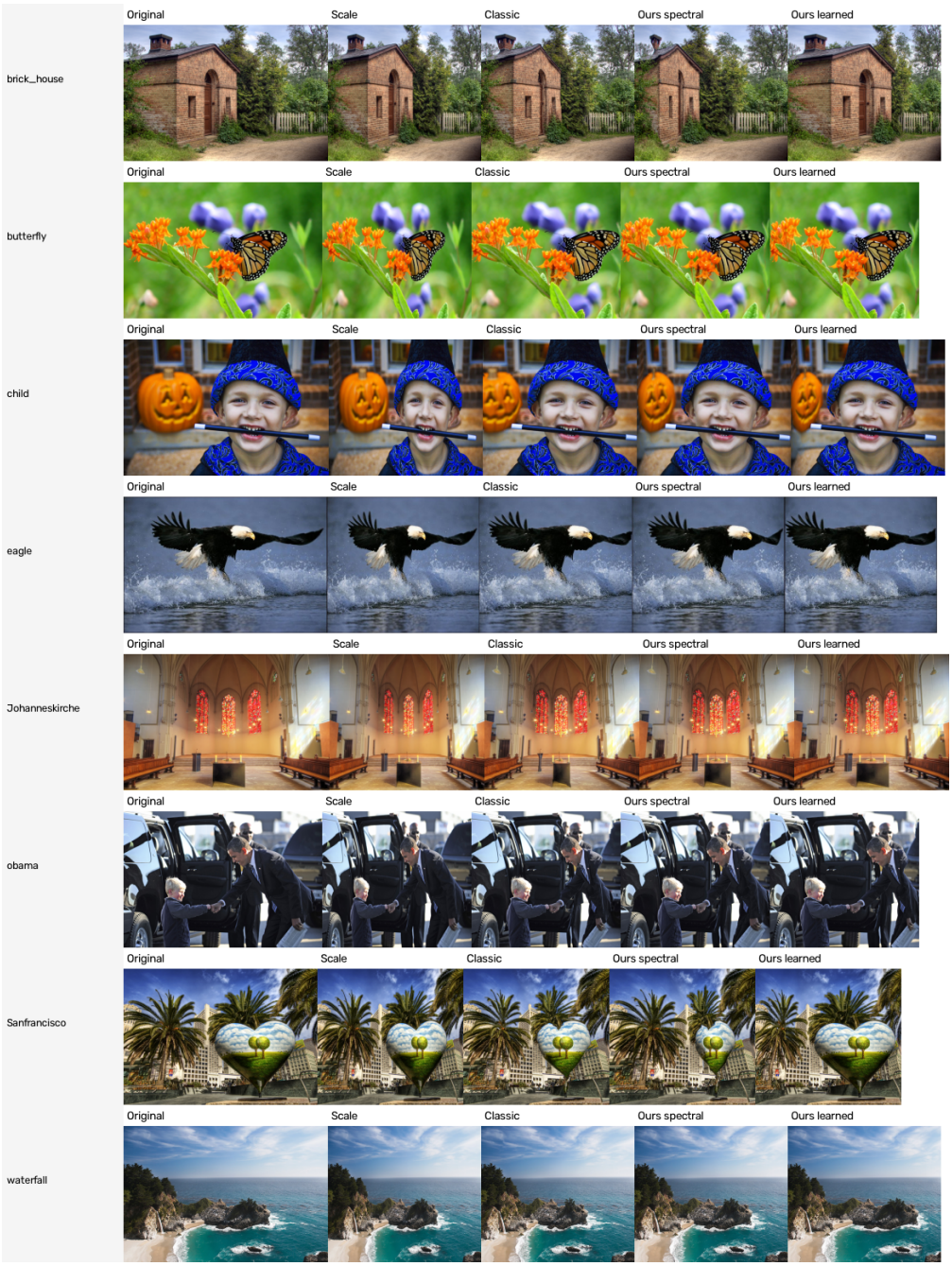

Figure 16. All eight original images at 75% width. Columns match Figure 15. These are impressions, not measurements. The images were not selected by outcome.

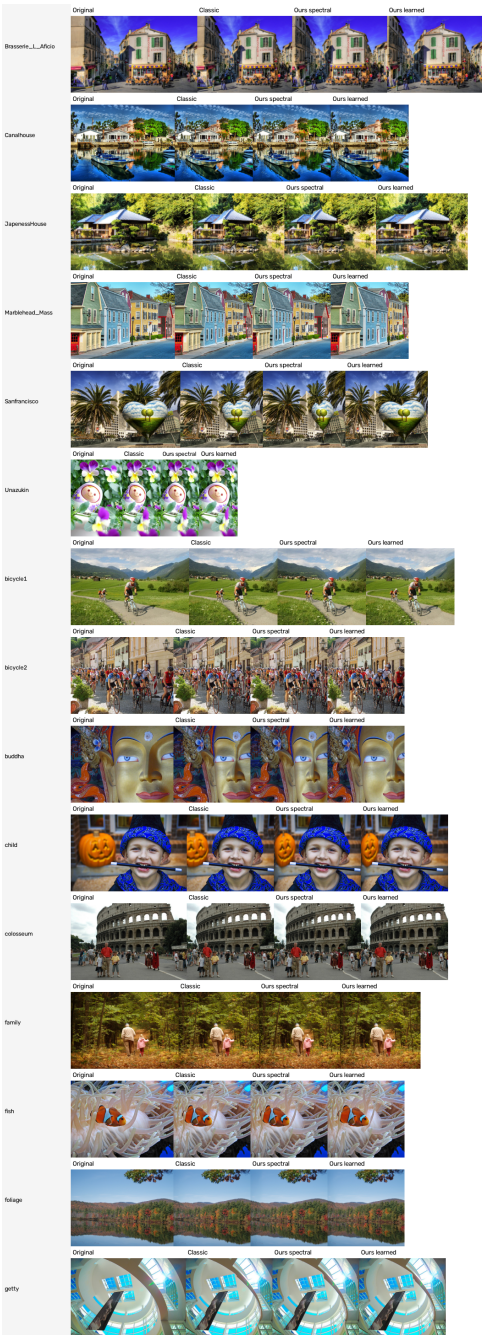

Figure 17. First 15 images of the seed-604 sample at 75% width, alphabetical order. Columns are the original, classic seams, Ours (spectral), and Ours (learned). These are impressions, not measurements. The images were not selected by outcome.

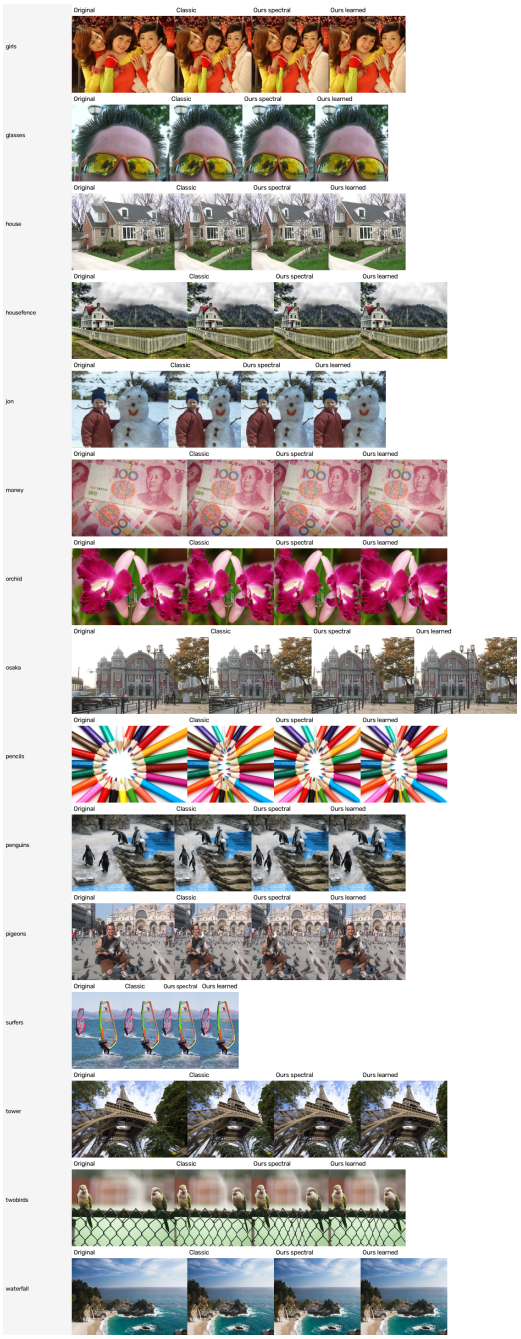

Figure 18. Remaining 15 images of the seed-604 sample at 75% width, continuing the alphabetical order of Figure 17. Columns match that figure. These are impressions, not measurements. The images were not selected by outcome.

E. Source code

The listings below are the program submitted with this report. They match the source included with this submission.

```
"""Spatial-domain image retargeting. We stay in the pixel grid and do not train a retargeting network."""

from .retarget import retarget

__all__ = ["retarget"]
```

Listing 1. src/__init__.py

```
"""Luminance, Hou and Zhang saliency, an optional U2-Net-small map, and oriented lines.

We compute saliency and the line maps once, then crop them with each seam. We recompute forward energy from the luminance that is left. We tried a curve map and chose not to put it in the cost.
"""

from __future__ import annotations

import cv2
import numpy as np


def luminance(image: np.ndarray) -> np.ndarray:
    """Rec. 601 luminance in ``[0, 1]``. ``image`` is RGB uint8 or float."""
    rgb = image.astype(np.float32)
    if rgb.max() > 1.0:
        rgb = rgb / 255.0
    if rgb.ndim == 2:
        return rgb
    r, g, b = rgb[:, :, 0], rgb[:, :, 1], rgb[:, :, 2]
    return 0.299 * r + 0.587 * g + 0.114 * b


def spectral_residual_saliency(lum: np.ndarray) -> np.ndarray:
    """Spectral residual in ``[0, 1]``. We subtract a 3x3 average from the log amplitude and keep the phase. We did not use a learned model for this map."""
    gray = lum.astype(np.float32)
    spectrum = np.fft.fft2(gray)
    log_amplitude = np.log(np.abs(spectrum) + 1e-8)
    phase = np.angle(spectrum)
    average = cv2.blur(log_amplitude.astype(np.float32), (3, 3))
    residual = log_amplitude - average
    reconstructed = np.fft.ifft2(np.exp(residual + 1j * phase))
    saliency = np.abs(reconstructed).astype(np.float32) ** 2
    saliency = cv2.GaussianBlur(saliency, (9, 9), 2.5)
    return _normalize(saliency)


_LEARNED_SESSION = None


def learned_saliency(rgb_uint8: np.ndarray) -> np.ndarray:
    """U2-Net-small through rembg ``u2netp``, in ``[0, 1]``. We load the checkpoint once and call it once per image, not on every seam."""
    global _LEARNED_SESSION
    rgb = np.ascontiguousarray(rgb_uint8)
    if rgb.ndim != 3 or rgb.shape[2] != 3 or rgb.dtype != np.uint8:
        raise ValueError("learned_saliency expects uint8 RGB of shape (H, W, 3)")
    if _LEARNED_SESSION is None:
        from rembg import new_session
        _LEARNED_SESSION = new_session("u2netp")
    from rembg import remove
    mask = np.asarray(remove(rgb, session=_LEARNED_SESSION, only_mask=True))
    if mask.ndim == 3:
        mask = mask[:, :, 0]
    height, width = rgb.shape[:2]
    if mask.shape[:2] != (height, width):
        mask = cv2.resize(mask.astype(np.float32), (width, height), interpolation=cv2.INTER_LINEAR)
    return _normalize(np.asarray(mask, dtype=np.float32))


def gradient_magnitude(lum: np.ndarray) -> np.ndarray:
    """Backward-energy map: sum of absolute Sobel responses."""
    gx = cv2.Sobel(lum.astype(np.float32), cv2.CV_32F, 1, 0, ksize=3)
    gy = cv2.Sobel(lum.astype(np.float32), cv2.CV_32F, 0, 1, ksize=3)
    return np.abs(gx) + np.abs(gy)
```

```
64
65
66  def curve_protection(lum: np.ndarray) -> np.ndarray:
67      """Strong edges in ``[0, 1]``, including curves. We computed this map and then chose not to use it in the cost."""
68      gray = np.clip(np.rint(lum * 255.0), 0, 255).astype(np.uint8)
69      blurred = cv2.GaussianBlur(gray, (0, 0), 2.2)
70      gx = cv2.Sobel(blurred, cv2.CV_32F, 1, 0, ksize=3)
71      gy = cv2.Sobel(blurred, cv2.CV_32F, 0, 1, ksize=3)
72      magnitude = np.sqrt(gx * gx + gy * gy).astype(np.float32)
73      cutoff = float(np.quantile(magnitude, 0.90))
74      magnitude[magnitude < cutoff] = 0.0
75      kernel = np.ones((3, 3), np.uint8)
76      magnitude = cv2.dilate(magnitude, kernel)
77      return _normalize(magnitude)
78
79
80  def line_protection(lum: np.ndarray, seam_direction: str, detector: str = "hough") -> np.ndarray:
81      """Line map in ``[0, 1]``. ``hough`` or ``lsd``. We keep segments of at least 30 px and 12% of the short side, and at most 40 of them.
82
83      We chose the weight from the seam direction: ``|sin theta|`` when the seam is vertical, ``|cos theta|`` when it is horizontal.
84      """
85      if seam_direction not in ("vertical", "horizontal"):
86          raise ValueError("seam_direction must be 'vertical' or 'horizontal'")
87
88      gray_u8 = np.clip(np.rint(lum * 255.0), 0, 255).astype(np.uint8)
89      blurred = cv2.GaussianBlur(gray_u8, (5, 5), 1.2)
90      height, width = lum.shape
91      min_length = max(30, int(round(0.12 * min(height, width))))
92      segments = _line_segments(blurred, min_length, detector)
93      protection = np.zeros((height, width), np.float32)
94      if len(segments) == 0:
95          return protection
96      ranked = []
97      for x1, y1, x2, y2 in segments:
98          length = float(np.hypot(x2 - x1, y2 - y1))
99          if length < min_length:
100             continue
101         ranked.append((length, int(round(x1)), int(round(y1)), int(round(x2)), int(round(y2))))
102     ranked.sort(reverse=True)
103     layer = np.zeros((height, width), np.float32)
104     for _, x1, y1, x2, y2 in ranked[:40]:
105         theta = np.arctan2(float(y2 - y1), float(x2 - x1))
106         if seam_direction == "vertical":
107             weight = abs(np.sin(theta))
108         else:
109             weight = abs(np.cos(theta))
110         if weight < 1e-3:
111             continue
112         layer.fill(0.0)
113         cv2.line(layer, (x1, y1), (x2, y2), float(weight), thickness=2)
114         protection = np.maximum(protection, layer)
115     return protection
116
117
118 def extra_energy(saliency: np.ndarray, lines: np.ndarray, w_s: float, w_l: float,
119                  curves: np.ndarray | None = None, w_c: float = 0.0) -> np.ndarray:
120     """The extra cost we add: saliency, oriented lines, and the curve map only if we turned it on."""
121     cost = w_s * saliency + w_l * lines
122     if curves is not None and w_c != 0.0:
123         cost = cost + w_c * curves
124     return cost.astype(np.float32)
125
126
127 def _line_segments(blurred_u8: np.ndarray, min_length: int, detector: str) -> np.ndarray:
128     if detector == "lsd":
129         raw = cv2.createLineSegmentDetector().detect(blurred_u8)[0]
130     elif detector == "hough":
131         edges = cv2.Canny(blurred_u8, 80, 160, apertureSize=3)
132         raw = cv2.HoughLinesP(
133             edges,
134             rho=1,
135             theta=np.pi / 180.0,
136             threshold=50,
137             minLineLength=min_length,
138             maxLineGap=10,
139         )
140     else:
141         raise ValueError("detector must be 'hough' or 'lsd'")
142     if raw is None or len(raw) == 0:
```

```
        return np.zeros((0, 4), np.float32)
    # We reshape both detectors to (N, 4). Hough is (N, 1, 4) on OpenCV 4 and (N, 4) on OpenCV 5. LSD is already (N, 4).
    return np.asarray(raw, dtype=np.float32).reshape(-1, 4)


def _normalize(values: np.ndarray) -> np.ndarray:
    values = values.astype(np.float32)
    low = float(values.min())
    high = float(values.max())
    if high - low < 1e-8:
        return np.zeros_like(values)
    return (values - low) / (high - low)
```

Listing 2. src/energy.py

```
"""Vertical-seam dynamic programs, removal, and insertion.

We run a horizontal seam by transposing the image and using this same code. We use Rubinstein's 2008 forward energy, and Avidan and Shamir's 2007 backward energy for the classic baseline.
"""

from __future__ import annotations

import numpy as np

from .energy import extra_energy, gradient_magnitude, luminance


def backward_seam(energy: np.ndarray) -> tuple[np.ndarray, float]:
    """Minimum seam under the classic backward cost.

    ``M(i, j) = E(i, j) + min(M(i-1, j-1), M(i-1, j), M(i-1, j+1))``.
    Returns the seam as one column index per row, and the seam cost.
    """
    height, width = energy.shape
    cumulative = np.zeros((height, width), np.float64)
    step = np.zeros((height, width), np.int8)
    cumulative[0] = energy[0]
    from_left = np.empty(width, np.float64)
    from_right = np.empty(width, np.float64)

    for i in range(1, height):
        previous = cumulative[i - 1]
        from_left[0] = np.inf
        from_left[1:] = previous[:-1]
        from_right[-1] = np.inf
        from_right[:-1] = previous[1:]
        which = np.ones(width, np.int8)  # 0 from j-1, 1 from j, 2 from j+1
        best = previous + 0.0
        better = from_left < best
        best = np.where(better, from_left, best)
        which = np.where(better, np.int8(0), which)
        better = from_right < best
        best = np.where(better, from_right, best)
        which = np.where(better, np.int8(2), which)
        cumulative[i] = energy[i] + best
        step[i] = which - 1

    return _backtrack(step, cumulative)


def forward_seam(lum: np.ndarray, pixel_cost: np.ndarray) -> tuple[np.ndarray, float]:
    """Forward energy plus ``pixel_cost``. We use the arrival costs C_U, C_L, and C_R, and we treat a step that leaves the image as infinite."""
    height, width = lum.shape
    cumulative = np.zeros((height, width), np.float64)
    step = np.zeros((height, width), np.int8)
    cumulative[0] = pixel_cost[0]
    left = np.empty(width, np.float64)
    right = np.empty(width, np.float64)
    from_left = np.empty(width, np.float64)
    from_right = np.empty(width, np.float64)

    for i in range(1, height):
        row = lum[i]
        above = lum[i - 1]
        left[0] = row[0]
        left[1:] = row[:-1]
        right[-1] = row[-1]
```

```
        right[:-1] = row[1:]

        cost_up = np.abs(right - left)
        cost_left = cost_up + np.abs(above - left)
        cost_right = cost_up + np.abs(above - right)

        previous = cumulative[i - 1]
        from_left[0] = np.inf
        from_left[1:] = previous[:-1] + cost_left[1:]
        from_right[-1] = np.inf
        from_right[:-1] = previous[1:] + cost_right[:-1]
        best = previous + cost_up
        which = np.ones(width, np.int8)
        better = from_left < best
        best = np.where(better, from_left, best)
        which = np.where(better, np.int8(0), which)
        better = from_right < best
        best = np.where(better, from_right, best)
        which = np.where(better, np.int8(2), which)
        cumulative[i] = pixel_cost[i] + best
        step[i] = which - 1

    return _backtrack(step, cumulative)


def vertical_seam(image: np.ndarray, saliency: np.ndarray, lines: np.ndarray,
                  mode: str, w_s: float, w_l: float,
                  curves: np.ndarray | None = None, w_c: float = 0.0) -> tuple[np.ndarray, float]:
    """One vertical seam on ``image`` using the matching line map."""
    lum = luminance(image)
    if mode == "classic":
        return backward_seam(gradient_magnitude(lum))
    if mode != "forward":
        raise ValueError("mode must be 'classic' or 'forward'")
    pixel_cost = extra_energy(saliency, lines, w_s, w_l, curves, w_c)
    return forward_seam(lum, pixel_cost)


def remove_vertical_seam(array: np.ndarray, seam: np.ndarray) -> np.ndarray:
    """We drop one pixel per row. This works for ``HxW``, ``HxWxC``, and index maps."""
    height, width = array.shape[:2]
    keep = np.ones((height, width), dtype=bool)
    keep[np.arange(height), seam] = False
    if array.ndim == 2:
        return array[keep].reshape(height, width - 1)
    return array[keep].reshape(height, width - 1, array.shape[2])


def insert_vertical_seams(image: np.ndarray, seams: list[np.ndarray]) -> np.ndarray:
    """We duplicate each recorded seam pixel and average it with its right neighbor.

    ``seams`` stores column indices into ``image`` itself (the picture before
    insertion), one array of length ``H`` per seam. Seams must not reuse a pixel.
    """
    if not seams:
        return image
    height, width = image.shape[:2]
    channels = 1 if image.ndim == 2 else image.shape[2]
    source = image.reshape(height, width, channels).astype(np.float32)
    order = np.sort(np.stack(seams, axis=1).astype(np.int32), axis=1)
    count = order.shape[1]
    output = np.empty((height, width + count, channels), np.float32)

    for row in range(height):
        cols = order[row]
        written = 0
        cursor = 0
        src_row = source[row]
        dst_row = output[row]
        for col in cols:
            run = int(col) - cursor + 1
            dst_row[written:written + run] = src_row[cursor:int(col) + 1]
            written += run
            neighbor = src_row[int(col) + 1] if int(col) + 1 < width else src_row[int(col)]
            dst_row[written] = 0.5 * (src_row[int(col)] + neighbor)
            written += 1
            cursor = int(col) + 1
        if cursor < width:
            dst_row[written:] = src_row[cursor:]
```

```
    result = np.clip(np.rint(output), 0, 255).astype(np.uint8)
    if image.ndim == 2:
        return result[:, :, 0]
    return result


def _backtrack(step: np.ndarray, cumulative: np.ndarray) -> tuple[np.ndarray, float]:
    height, width = step.shape
    seam = np.zeros(height, np.int32)
    seam[-1] = int(np.argmin(cumulative[-1]))
    cost = float(cumulative[-1, seam[-1]])
    for i in range(height - 1, 0, -1):
        seam[i - 1] = int(np.clip(seam[i] + int(step[i, seam[i]]), 0, width - 1))
    return seam, cost
```

Listing 3. src/seam.py

```
"""Uniform scaling and center-crop. We use these as baselines that ignore seam cost."""

from __future__ import annotations

import cv2
import numpy as np


def scale_to(image: np.ndarray, target_h: int, target_w: int) -> np.ndarray:
    """Resize to the exact target. Area interpolation when shrinking."""
    height, width = image.shape[:2]
    shrinking = target_h <= height and target_w <= width
    interpolation = cv2.INTER_AREA if shrinking else cv2.INTER_LINEAR
    return cv2.resize(image, (target_w, target_h), interpolation=interpolation)


def center_crop_resize(image: np.ndarray, target_h: int, target_w: int) -> np.ndarray:
    """Crop the centered window with the target aspect, then resize to it."""
    height, width = image.shape[:2]
    target_aspect = target_w / float(target_h)
    aspect = width / float(height)
    if aspect > target_aspect:
        new_w = max(1, int(round(height * target_aspect)))
        x0 = max(0, (width - new_w) // 2)
        cropped = image[:, x0:x0 + new_w]
    else:
        new_h = max(1, int(round(width / target_aspect)))
        y0 = max(0, (height - new_h) // 2)
        cropped = image[y0:y0 + new_h, :]
    if cropped.size == 0:
        cropped = image
    return scale_to(cropped, target_h, target_w)


def limit_long_side(image: np.ndarray, max_side: int = 640) -> np.ndarray:
    """We shrink so the long side is at most ``max_side``. We never enlarge here."""
    height, width = image.shape[:2]
    long_side = max(height, width)
    if long_side <= max_side:
        return np.ascontiguousarray(image)
    scale = max_side / float(long_side)
    new_w = max(2, int(round(width * scale)))
    new_h = max(2, int(round(height * scale)))
    return scale_to(image, new_h, new_w)
```

Listing 4. src/baselines.py

```
"""Retarget one RGB image to an exact height and width.

We resize for ``scale`` and keep the centered window for ``crop``. ``classic`` is backward energy. For ``ours`` we add saliency and Hough lines to forward energy, with ``weights=(w_s, w_l)``. For ``hybrid`` we use that same cost, stop the seams at 75% of a side, and scale the rest.
"""

from __future__ import annotations

import numpy as np

from .baselines import center_crop_resize, scale_to
from .energy import (
    curve_protection,
```

```
    learned_saliency,
    line_protection,
    luminance,
    spectral_residual_saliency,
)
from .metrics import accumulate_seam, empty_counts
from .seam import insert_vertical_seams, remove_vertical_seam, vertical_seam

DEFAULT_WEIGHTS = (0.5, 0.8)
# We chose 0.25 for the learned source only. It was the smallest of {0.25, 0.5, 1, 2, 4} that kept the learned mass under half the forward-only value on Johanneskirche and obama, without a tear. Spectral runs still use DEFAULT_WEIGHTS.
LEARNED_W_S = 0.25
LEARNED_W_L = 0.8
# We let the seams remove at most this fraction of a side, then we scale.
HYBRID_KEEP = 0.75


def retarget(image: np.ndarray, target_hw: tuple[int, int], method: str = "ours",
             weights: tuple[float, float] = DEFAULT_WEIGHTS, return_info: bool = False,
             saliency_source: str = "spectral", score_both: bool = False):
    """RGB uint8 image. With ``return_info`` we also return the seam counts. Scale and crop have no seams, and for hybrid we score only the seams we actually cut."""
    rgb = _as_uint8_rgb(image)
    target_h = max(2, int(target_hw[0]))
    target_w = max(2, int(target_hw[1]))
    w_s, w_l = float(weights[0]), float(weights[1])

    if method == "scale":
        out = scale_to(rgb, target_h, target_w)
        info = _baseline_info(out, (target_h, target_w))
        return (out, info) if return_info else out
    if method == "crop":
        out = center_crop_resize(rgb, target_h, target_w)
        info = _baseline_info(out, (target_h, target_w))
        return (out, info) if return_info else out
    if method not in ("classic", "ours", "hybrid"):
        raise ValueError("method must be 'scale', 'crop', 'classic', 'ours', or 'hybrid'")

    mode = "classic" if method == "classic" else "forward"
    height, width = rgb.shape[:2]
    if method == "hybrid":
        # We use the same cost as ours, and we stop the seams at 75% of the original side.
        shrink_h = _hybrid_stop(height, target_h)
        shrink_w = _hybrid_stop(width, target_w)
        detector, w_c = "hough", 0.0
    else:
        shrink_h, shrink_w = min(target_h, height), min(target_w, width)
        detector, w_c = "hough", 0.0
    carved, counts = _shrink(
        rgb, shrink_h, shrink_w, mode, w_s, w_l, w_c, detector, saliency_source, score_both,
    )
    counts["scaled_after_seams"] = 0
    scale_h = target_h if target_h < carved.shape[0] else carved.shape[0]
    scale_w = target_w if target_w < carved.shape[1] else carved.shape[1]
    if (scale_h, scale_w) != carved.shape[:2]:
        carved = scale_to(carved, scale_h, scale_w)
        counts["scaled_after_seams"] = 1

    if target_w > carved.shape[1]:
        carved, insert_counts = _enlarge_width(
            carved, target_w, mode, w_s, w_l, w_c, detector, saliency_source,
        )
        _merge_path_counts(counts, insert_counts)
    if target_h > carved.shape[0]:
        carved, insert_counts = _enlarge_height(
            carved, target_h, mode, w_s, w_l, w_c, detector, saliency_source,
        )
        _merge_path_counts(counts, insert_counts)

    info = dict(counts)
    info["output_hw"] = tuple(carved.shape[:2])
    info["target_hw"] = (target_h, target_w)
    return (carved, info) if return_info else carved


def next_vertical_seam(image: np.ndarray, method: str = "ours",
                       weights: tuple[float, float] = DEFAULT_WEIGHTS) -> np.ndarray:
    """Column index per row of the next vertical seam, for the demo overlay."""
    rgb = _as_uint8_rgb(image)
```

```
    maps = protection_maps(rgb)
    mode = "classic" if method == "classic" else "forward"
    w_s, w_l = (0.0, 0.0) if method == "classic" else (float(weights[0]), float(weights[1]))
    seam, _ = vertical_seam(rgb, maps["saliency"], maps["lines_vertical"], mode, w_s, w_l)
    return seam


def protection_maps(image: np.ndarray) -> dict[str, np.ndarray]:
    """Luminance, saliency, and both line maps, for the demo overlays."""
    rgb = _as_uint8_rgb(image)
    lum = luminance(rgb)
    return {
        "luminance": lum,
        "saliency": spectral_residual_saliency(lum),
        "lines_vertical": line_protection(lum, "vertical"),
        "lines_horizontal": line_protection(lum, "horizontal"),
    }


def _hybrid_stop(original: int, target: int, keep: float = HYBRID_KEEP) -> int:
    """The size we still reach with seams. We finish any deeper cut by scaling."""
    if target >= original:
        return original
    floor = max(2, int(round(original * keep)))
    return max(target, floor)


def _drop_curve(curves, seam, transpose: bool):
    if curves is None:
        return None
    if transpose:
        return remove_vertical_seam(curves.T, seam).T
    return remove_vertical_seam(curves, seam)


def _saliency_for_cost(image, lum, source: str):
    if source == "spectral":
        return spectral_residual_saliency(lum)
    if source == "learned":
        return learned_saliency(image)
    raise ValueError("saliency_source must be 'spectral' or 'learned'")


def _count_mass(counts: dict, key: str, saliency: np.ndarray, seam: np.ndarray) -> None:
    rows = np.arange(seam.shape[0])
    counts[key] += float(saliency[rows, seam].sum())


def _count_blob(counts: dict, saliency: np.ndarray, seam: np.ndarray) -> None:
    """Seam pixels whose learned-map value is above 0.5. We skip this unless we armed that count."""
    if "learned_blob_hits" not in counts:
        return
    rows = np.arange(seam.shape[0])
    counts["learned_blob_hits"] += int(np.count_nonzero(saliency[rows, seam] > 0.5))


def _shrink(image, target_h, target_w, mode, w_s, w_l, w_c=0.0, detector="hough",
            saliency_source="spectral", score_both=False):
    lum = luminance(image)
    saliency = _saliency_for_cost(image, lum, saliency_source)
    line_v = line_protection(lum, "vertical", detector)
    line_h = line_protection(lum, "horizontal", detector)
    # We score LSD only. We chose not to pass it into the dynamic program.
    lsd_v = line_protection(lum, "vertical", "lsd")
    lsd_h = line_protection(lum, "horizontal", "lsd")
    curves = curve_protection(lum) if w_c else None
    counts = empty_counts()
    counts["saliency_total"] = float(saliency.sum())
    if saliency_source == "learned" or (score_both and saliency_source == "spectral"):
        counts["learned_blob_hits"] = 0
    # We score the other saliency map only. We chose not to pass it into the dynamic program.
    other = None
    other_key = None
    if score_both:
        if saliency_source == "spectral":
            other = learned_saliency(image)
            other_key = "learned_removed"
            counts["learned_total"] = float(other.sum())
            counts["learned_removed"] = 0.0
```

```
        else:
            other = spectral_residual_saliency(lum)
            other_key = "spectral_removed"
            counts["spectral_total"] = float(other.sum())
            counts["spectral_removed"] = 0.0

    current = image
    while current.shape[0] > target_h or current.shape[1] > target_w:
        can_v = current.shape[1] > target_w and current.shape[1] > 2
        can_h = current.shape[0] > target_h and current.shape[0] > 2
        if not can_v and not can_h:
            break
        if can_v and can_h:
            seam_v, cost_v = vertical_seam(current, saliency, line_v, mode, w_s, w_l, curves, w_c)
            seam_h, cost_h = _horizontal_seam(current, saliency, line_h, mode, w_s, w_l, curves, w_c)
            take_vertical = cost_v <= cost_h
        else:
            take_vertical = can_v
            seam_v = seam_h = None

        if take_vertical:
            if seam_v is None:
                seam_v, _ = vertical_seam(current, saliency, line_v, mode, w_s, w_l, curves, w_c)
            accumulate_seam(counts, saliency, line_v, seam_v, vertical=True)
            _count_lsd(counts, lsd_v, seam_v)
            if saliency_source == "learned":
                _count_blob(counts, saliency, seam_v)
            elif other is not None:
                _count_blob(counts, other, seam_v)
            if other is not None:
                _count_mass(counts, other_key, other, seam_v)
                other = remove_vertical_seam(other, seam_v)
            current = remove_vertical_seam(current, seam_v)
            saliency = remove_vertical_seam(saliency, seam_v)
            line_v = remove_vertical_seam(line_v, seam_v)
            line_h = remove_vertical_seam(line_h, seam_v)
            lsd_v = remove_vertical_seam(lsd_v, seam_v)
            lsd_h = remove_vertical_seam(lsd_h, seam_v)
            curves = _drop_curve(curves, seam_v, transpose=False)
        else:
            if seam_h is None:
                seam_h, _ = _horizontal_seam(current, saliency, line_h, mode, w_s, w_l, curves, w_c)
            transposed_sal = saliency.T
            transposed_line = line_h.T
            accumulate_seam(counts, transposed_sal, transposed_line, seam_h, vertical=False)
            _count_lsd(counts, lsd_h.T, seam_h)
            if saliency_source == "learned":
                _count_blob(counts, transposed_sal, seam_h)
            elif other is not None:
                _count_blob(counts, other.T, seam_h)
            if other is not None:
                _count_mass(counts, other_key, other.T, seam_h)
                other = remove_vertical_seam(other.T, seam_h).T
            current = _transpose_image(remove_vertical_seam(_transpose_image(current), seam_h))
            saliency = remove_vertical_seam(transposed_sal, seam_h).T
            line_h = remove_vertical_seam(transposed_line, seam_h).T
            line_v = remove_vertical_seam(line_v.T, seam_h).T
            lsd_h = remove_vertical_seam(lsd_h.T, seam_h).T
            lsd_v = remove_vertical_seam(lsd_v.T, seam_h).T
            curves = _drop_curve(curves, seam_h, transpose=True)
    if score_both and saliency_source == "spectral":
        counts["spectral_total"] = counts["saliency_total"]
        counts["spectral_removed"] = counts["saliency_removed"]
    elif score_both:
        counts["learned_total"] = counts["saliency_total"]
        counts["learned_removed"] = counts["saliency_removed"]
    return current, counts


def _enlarge_width(image, target_w, mode, w_s, w_l, w_c=0.0, detector="hough",
                   saliency_source="spectral"):
    """We record seams by removing them, then we insert averaged duplicates back into ``image``."""
    current = image
    total = empty_counts()
    while current.shape[1] < target_w:
        room = current.shape[1] - 2
        need = target_w - current.shape[1]
        step = min(need, room)
        if step <= 0:
```

```
            break
        seams, counts = _record_vertical_seams(
            current, step, mode, w_s, w_l, w_c, detector, saliency_source,
        )
        _merge_path_counts(total, counts)
        current = insert_vertical_seams(current, seams)
    return current, total


def _enlarge_height(image, target_h, mode, w_s, w_l, w_c=0.0, detector="hough",
                    saliency_source="spectral"):
    transposed = _transpose_image(image)
    enlarged, counts = _enlarge_width(
        transposed, target_h, mode, w_s, w_l, w_c, detector, saliency_source,
    )
    counts["horizontal_seams"] = counts["vertical_seams"]
    counts["vertical_seams"] = 0
    return _transpose_image(enlarged), counts


def _record_vertical_seams(image, count, mode, w_s, w_l, w_c=0.0, detector="hough",
                           saliency_source="spectral"):
    height, width = image.shape[:2]
    lum = luminance(image)
    saliency = _saliency_for_cost(image, lum, saliency_source)
    lines = line_protection(lum, "vertical", detector)
    curves = curve_protection(lum) if w_c else None
    columns = np.tile(np.arange(width, dtype=np.int32), (height, 1))
    work = image
    seams = []
    counts = empty_counts()
    counts["saliency_total"] = float(saliency.sum())
    for _ in range(count):
        seam, _ = vertical_seam(work, saliency, lines, mode, w_s, w_l, curves, w_c)
        rows = np.arange(work.shape[0])
        seams.append(columns[rows, seam].copy())
        # We keep the pixel when we insert a seam, so we do not count that saliency as removed.
        counts["line_hits"] += int(np.count_nonzero(lines[rows, seam] > 0))
        counts["seam_pixels"] += int(seam.shape[0])
        counts["vertical_seams"] += 1
        work = remove_vertical_seam(work, seam)
        saliency = remove_vertical_seam(saliency, seam)
        lines = remove_vertical_seam(lines, seam)
        curves = _drop_curve(curves, seam, transpose=False)
        columns = remove_vertical_seam(columns, seam)
    return seams, counts


def _horizontal_seam(image, saliency, line_h, mode, w_s, w_l, curves=None, w_c=0.0):
    curve_t = None if curves is None else curves.T
    return vertical_seam(
        _transpose_image(image), saliency.T, line_h.T, mode, w_s, w_l, curve_t, w_c,
    )


def _transpose_image(image: np.ndarray) -> np.ndarray:
    if image.ndim == 2:
        return np.ascontiguousarray(np.transpose(image, (1, 0)))
    return np.ascontiguousarray(np.transpose(image, (1, 0, 2)))


def _count_lsd(counts: dict, lsd: np.ndarray, seam: np.ndarray) -> None:
    rows = np.arange(seam.shape[0])
    counts["lsd_hits"] += int(np.count_nonzero(lsd[rows, seam] > 0))


def _merge_path_counts(total: dict, extra: dict) -> None:
    for key in ("line_hits", "lsd_hits", "seam_pixels", "vertical_seams", "horizontal_seams"):
        total[key] += extra[key]


def _baseline_info(image, target_hw) -> dict:
    counts = empty_counts()
    counts["output_hw"] = tuple(image.shape[:2])
    counts["target_hw"] = target_hw
    return counts


def _as_uint8_rgb(image: np.ndarray) -> np.ndarray:
```

```
    array = np.asarray(image)
    if array.ndim == 2:
        array = np.stack([array, array, array], axis=-1)
    if array.shape[2] == 4:
        array = array[:, :, :3]
    if array.dtype != np.uint8:
        values = array.astype(np.float32)
        if values.max() <= 1.0:
            values = values * 255.0
        array = np.clip(np.rint(values), 0, 255).astype(np.uint8)
    return np.ascontiguousarray(array)
```

Listing 5. src/retarget.py

```
"""Seam scores. We measure saliency as deleted map mass over total mass, and lines as hits per 100 seam pixels. Scale and crop have neither score, so we leave both unset."""

from __future__ import annotations

import numpy as np


def empty_counts() -> dict:
    return {
        "saliency_total": 0.0,
        "saliency_removed": 0.0,
        "line_hits": 0,
        "lsd_hits": 0,
        "seam_pixels": 0,
        "vertical_seams": 0,
        "horizontal_seams": 0,
    }


def saliency_removed_fraction(counts: dict) -> float:
    total = float(counts["saliency_total"])
    if total <= 0.0:
        return 0.0
    return float(counts["saliency_removed"]) / total


def line_crossings_per_100(counts: dict) -> float:
    pixels = int(counts["seam_pixels"])
    if pixels <= 0:
        return 0.0
    return 100.0 * float(counts["line_hits"]) / pixels


def lsd_crossings_per_100(counts: dict) -> float:
    """Seam pixels on the LSD map, per 100 seam pixels. We use this as a check. We did not use it to choose seams."""
    pixels = int(counts["seam_pixels"])
    if pixels <= 0:
        return 0.0
    return 100.0 * float(counts.get("lsd_hits", 0)) / pixels


def size_matches(image: np.ndarray, target_hw: tuple[int, int]) -> bool:
    return tuple(image.shape[:2]) == (int(target_hw[0]), int(target_hw[1]))


def accumulate_seam(counts: dict, saliency: np.ndarray, lines: np.ndarray,
                    seam: np.ndarray, vertical: bool) -> None:
    rows = np.arange(seam.shape[0])
    counts["saliency_removed"] += float(saliency[rows, seam].sum())
    counts["line_hits"] += int(np.count_nonzero(lines[rows, seam] > 0))
    counts["seam_pixels"] += int(seam.shape[0])
    if vertical:
        counts["vertical_seams"] += 1
    else:
        counts["horizontal_seams"] += 1
```

Listing 6. src/metrics.py

```
"""RGB overlays. We use these in the demo and in the experiment figures."""

from __future__ import annotations

```

```
import cv2
import numpy as np


def colorize(values: np.ndarray) -> np.ndarray:
    """Map a ``[0, 1]`` array to an RGB image with the viridis colormap."""
    scaled = np.clip(np.rint(values * 255.0), 0, 255).astype(np.uint8)
    bgr = cv2.applyColorMap(scaled, cv2.COLORMAP_VIRIDIS)
    return cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)


def draw_vertical_seam(image: np.ndarray, seam: np.ndarray, color=(220, 40, 40)) -> np.ndarray:
    """Paint a vertical seam, two pixels wide, on a copy of ``image``."""
    painted = image.copy()
    height, width = painted.shape[:2]
    rows = np.arange(height)
    cols = np.clip(seam, 0, width - 1)
    painted[rows, cols] = color
    painted[rows, np.clip(cols - 1, 0, width - 1)] = color
    return painted
```

Listing 7. src/display.py

```
"""Gradio demo. We compare scale, crop, classic seams, forward energy, and hybrid. We cap the long side at 640 px so a result can be
    recomputed on request. ``w_s`` and ``w_l`` are the saliency and line weights."""

from __future__ import annotations

import time
from pathlib import Path

import gradio as gr
import numpy as np

from src.baselines import limit_long_side
from src.display import colorize, draw_vertical_seam
from src.metrics import line_crossings_per_100, saliency_removed_fraction
from src.retarget import next_vertical_seam, protection_maps, retarget

ROOT = Path(__file__).resolve().parent
IMAGE_DIR = ROOT / "experiments" / "retargetme"
ROLES_PATH = ROOT / "experiments" / "results" / "roles.txt"
MAX_SIDE = 640


def _find_named(stem: str) -> Path | None:
    for suffix in (".png", ".jpg", ".jpeg", ".PNG", ".JPG"):
        candidate = IMAGE_DIR / f"{stem}{suffix}"
        if candidate.exists():
            return candidate
    return None


def example_paths() -> list[str]:
    """We prefer the line, saliency, and failure images we chose for the experiments."""
    ordered: list[Path] = []
    if ROLES_PATH.exists():
        roles = {}
        for line in ROLES_PATH.read_text(encoding="utf-8").splitlines():
            if "=" in line:
                key, value = line.split("=", 1)
                roles[key.strip()] = value.strip()
        for key in ("lines", "saliency", "failure"):
            path = _find_named(roles.get(key, ""))
            if path is not None:
                ordered.append(path)
    if not ordered and IMAGE_DIR.exists():
        ordered = [
            path for path in sorted(IMAGE_DIR.iterdir())
            if path.suffix.lower() in {".png", ".jpg", ".jpeg"}
        ][:3]
    return [str(path) for path in ordered]


def _fit_height(image: np.ndarray, height: int) -> np.ndarray:
    scale = height / float(image.shape[0])
    width = max(1, int(round(image.shape[1] * scale)))
    import cv2
    return cv2.resize(image, (width, height), interpolation=cv2.INTER_AREA)
```

```

def _hstack(images: list[np.ndarray]) -> np.ndarray:
    height = max(image.shape[0] for image in images)
    fitted = [_fit_height(image, height) for image in images]
    gap = np.full((height, 8, 3), 255, np.uint8)
    pieces: list[np.ndarray] = []
    for index, image in enumerate(fitted):
        if index:
            pieces.append(gap)
        pieces.append(image)
    return np.hstack(pieces)


def retarget_image(image, width_percent, height_percent, method, w_s, w_l,
                   show_saliency, show_lines, show_seam):
    if image is None:
        return None, None, "Upload an image or pick an example."
    rgb = limit_long_side(np.asarray(image), MAX_SIDE)
    height, width = rgb.shape[:2]
    target_h = max(2, int(round(height * float(height_percent) / 100.0)))
    target_w = max(2, int(round(width * float(width_percent) / 100.0)))
    started = time.perf_counter()
    output, info = retarget(
        rgb, (target_h, target_w), method=method, weights=(float(w_s), float(w_l)), return_info=True,
    )
    elapsed = time.perf_counter() - started
    pair = _hstack([rgb, output])

    overlays = []
    if show_saliency or show_lines or show_seam:
        maps = protection_maps(rgb)
        if show_saliency:
            overlays.append(colorize(maps["saliency"]))
        if show_lines:
            overlays.append(colorize(maps["lines_vertical"]))
        if show_seam and method in ("classic", "ours", "hybrid"):
            seam = next_vertical_seam(rgb, method=method, weights=(float(w_s), float(w_l)))
            overlays.append(draw_vertical_seam(rgb, seam))
    overlay_image = _hstack(overlays) if overlays else None

    message = (
        f"Working size {height} x {width} -> {output.shape[0]} x {output.shape[1]} "
        f"in {elapsed:.1f}s. Method: {method}."
    )
    if method in ("classic", "ours", "hybrid") and info.get("seam_pixels", 0) > 0:
        message += (
            f" Saliency mass removed: {saliency_removed_fraction(info):.3f}."
            f" Line pixels per 100 seam pixels: {line_crossings_per_100(info):.1f}."
        )
    if info.get("scaled_after_seams"):
        message += " Seams stopped at 75% of the original side; scaling finished the cut."
    return pair, overlay_image, message


def build() -> gr.Blocks:
    with gr.Blocks(title="Image retargeting") as demo:
        gr.Markdown(
            """
# Image retargeting

Spatial-domain seam carving. **Ours** adds spectral-residual saliency and an
orientation-aware line map to Rubinstein forward energy. **Hybrid** uses
that same cost, carves only down to 75% of the original side, and scales the rest.

Suggested demo: a facade (turn the line weight up), a person (saliency),
then San Francisco at 50% width, comparing ours with hybrid.
            """
        )
        with gr.Row():
            source = gr.Image(type="numpy", label="Input", height=360)
            with gr.Column():
                width_percent = gr.Slider(50, 150, value=75, step=1, label="Target width (%)")
                height_percent = gr.Slider(50, 150, value=100, step=1, label="Target height (%)")
                method = gr.Radio(
                    ["hybrid", "ours", "classic", "scale", "crop"],
                    value="hybrid",
                    label="Method",
                )
```

```
                w_s = gr.Slider(0.0, 2.0, value=0.5, step=0.1, label="Saliency weight w_s")
                w_l = gr.Slider(0.0, 2.0, value=0.8, step=0.1, label="Line weight w_l")
                show_saliency = gr.Checkbox(value=True, label="Show saliency")
                show_lines = gr.Checkbox(value=True, label="Show vertical-seam line map")
                show_seam = gr.Checkbox(value=True, label="Show the next vertical seam")
                run = gr.Button("Retarget", variant="primary")
        result = gr.Image(type="numpy", label="Original (left) and retargeted (right)")
        maps = gr.Image(type="numpy", label="Saliency, line map, next seam")
        status = gr.Textbox(label="Notes", lines=2)
        run.click(
            retarget_image,
            inputs=[source, width_percent, height_percent, method, w_s, w_l,
                    show_saliency, show_lines, show_seam],
            outputs=[result, maps, status],
        )
        paths = example_paths()
        if paths:
            gr.Examples(examples=[[path] for path in paths], inputs=[source], label="Examples")
    return demo


if __name__ == "__main__":
    build().launch()
```

Listing 8. app.py

```
"""We download the RetargetMe source-image zip and keep eight images. We do not commit the zip itself."""

from __future__ import annotations

import re
import sys
import urllib.request
import zipfile
from pathlib import Path

PAGE = "https://people.csail.mit.edu/mrub/retargetme/download.html"
_OPENER = urllib.request.build_opener()
_OPENER.addheaders = [("User-Agent", "Mozilla/5.0 (compatible; ee604-retarget/1.0)")]
urllib.request.install_opener(_OPENER)
ROOT = Path(__file__).resolve().parents[1]
DEST = ROOT / "experiments" / "retargetme"
ZIP_PATH = ROOT / "experiments" / "retargetme_source.zip"

# We chose these stems from the RetargetMe archive: people, buildings, an animal, an object, a landscape, and a street.
KEEP = (
    "obama.png",
    "child.png",
    "brick_house.png",
    "Johanneskirche.png",
    "eagle.png",
    "butterfly.png",
    "waterfall.png",
    "Sanfrancisco.png",
)


def _source_zip_url() -> str:
    with urllib.request.urlopen(PAGE, timeout=60) as response:
        html = response.read().decode("utf-8", "replace")
    anchors = re.findall(r'<a[^>]+href="([^"]+)"[^>]*>(.*?)</a>', html, flags=re.I | re.S)
    candidates = []
    for href, raw_text in anchors:
        text = re.sub(r"<[^>]+>", "", raw_text)
        text = re.sub(r"\s+", " ", text).strip().lower()
        if href.lower().endswith(".zip"):
            candidates.append((text, urllib.request.urljoin(PAGE, href)))
            print(f"zip link: {text} -> {candidates[-1][1]}")
    for text, url in candidates:
        if "source image" in text and "only" in text:
            return url
    for text, url in candidates:
        if "source" in text and "retarget" not in text:
            return url
    raise RuntimeError("Could not find the source-image zip on the RetargetMe download page")


def main() -> None:
    DEST.mkdir(parents=True, exist_ok=True)
```

```
    present = {path.name for path in DEST.iterdir() if path.is_file()}
    if set(KEEP).issubset(present):
        print(f"subset already in {DEST}")
        return
    if not ZIP_PATH.exists():
        url = _source_zip_url()
        print(f"downloading {url}")
        urllib.request.urlretrieve(url, ZIP_PATH)
    with zipfile.ZipFile(ZIP_PATH) as archive:
        by_name = {Path(name).name: name for name in archive.namelist()}
        missing = [name for name in KEEP if name not in by_name]
        if missing:
            raise RuntimeError(f"missing from archive: {missing}")
        for stale in list(DEST.iterdir()):
            if stale.is_file() and stale.name not in KEEP:
                stale.unlink()
        for name in KEEP:
            (DEST / name).write_bytes(archive.read(by_name[name]))
            print("kept", name)
    print(f"wrote {len(KEEP)} images to {DEST}")


if __name__ == "__main__":
    try:
        main()
    except Exception as exc:
        print(f"download failed: {exc}", file=sys.stderr)
        sys.exit(1)
```

Listing 9. experiments/download_retargetme.py

```
"""We compare scale, crop, classic seams, and our model on the eight RetargetMe images. This writes the CSV and the contact sheets. We cap the long side at 640 px."""

from __future__ import annotations

import csv
import sys
import time
from pathlib import Path

import cv2
import numpy as np

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from src.baselines import limit_long_side  # noqa: E402
from src.display import colorize, draw_vertical_seam  # noqa: E402
from src.energy import line_protection, luminance  # noqa: E402
from src.metrics import (  # noqa: E402
    line_crossings_per_100,
    lsd_crossings_per_100,
    saliency_removed_fraction,
    size_matches,
)
from src.retarget import next_vertical_seam, protection_maps, retarget  # noqa: E402

IMAGE_DIR = ROOT / "experiments" / "retargetme"
RESULT_DIR = ROOT / "experiments" / "results"
MAX_SIDE = 640
PANEL_H = 168
WEIGHTS = {
    "forward": (0.0, 0.0),
    "forward_saliency": (0.5, 0.0),
    "forward_lines": (0.0, 0.8),
    "ours": (0.5, 0.8),
    "classic": (0.0, 0.0),
    "scale": (0.0, 0.0),
    "crop": (0.0, 0.0),
}
SEAM_METHODS = {"classic", "ours", "forward", "forward_saliency", "forward_lines"}
# We keep the contact sheets on this set. We chose not to draw every image when a CSV run is larger.
PAPER_FIGURES = (
    "brick_house",
    "butterfly",
    "child",
    "eagle",
    "Johanneskirche",
```

```
    "obama",
    "Sanfrancisco",
    "waterfall",
)


def load_rgb(path: Path) -> np.ndarray:
    bgr = cv2.imread(str(path), cv2.IMREAD_COLOR)
    if bgr is None:
        raise FileNotFoundError(path)
    return cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)


def discover_images() -> list[Path]:
    paths = sorted(
        [p for p in IMAGE_DIR.iterdir() if p.suffix.lower() in {".png", ".jpg", ".jpeg"}]
    )
    if len(paths) < 1:
        raise FileNotFoundError(
            f"No images in {IMAGE_DIR}. Run experiments/download_retargetme.py first."
        )
    return paths


def run_one(image: np.ndarray, target_hw, method: str) -> tuple[np.ndarray, dict, float]:
    weights = WEIGHTS[method]
    started = time.perf_counter()
    if method in ("forward", "forward_saliency", "forward_lines"):
        output, info = retarget(image, target_hw, method="ours", weights=weights, return_info=True)
    else:
        output, info = retarget(image, target_hw, method=method, weights=weights, return_info=True)
    elapsed = time.perf_counter() - started
    return output, info, elapsed


def row_for(name, setting, method, image, output, info, elapsed, target_hw) -> dict:
    seam = method in SEAM_METHODS or method in ("forward", "forward_saliency", "forward_lines")
    sal = ""
    lines = ""
    lsd = ""
    if seam and info.get("seam_pixels", 0) > 0:
        sal = f"{saliency_removed_fraction(info):.6f}"
        lines = f"{line_crossings_per_100(info):.4f}"
        lsd = f"{lsd_crossings_per_100(info):.4f}"
    elif seam:
        sal = "0"
        lines = "0"
        lsd = "0"
    return {
        "image": name,
        "setting": setting,
        "method": method,
        "w_s": WEIGHTS[method][0],
        "w_l": WEIGHTS[method][1],
        "target_h": target_hw[0],
        "target_w": target_hw[1],
        "out_h": output.shape[0],
        "out_w": output.shape[1],
        "size_ok": int(size_matches(output, target_hw)),
        "saliency_removed_fraction": sal,
        "line_crossings_per_100": lines,
        "runtime_s": f"{elapsed:.4f}",
        "lsd_crossings_per_100": lsd,
    }


def fit_panel(image: np.ndarray, title: str) -> np.ndarray:
    height, width = image.shape[:2]
    new_w = max(1, int(round(width * (PANEL_H / float(height)))))
    resized = cv2.resize(image, (new_w, PANEL_H), interpolation=cv2.INTER_AREA)
    bar = np.full((26, new_w, 3), 255, np.uint8)
    cv2.putText(bar, title, (4, 18), cv2.FONT_HERSHEY_SIMPLEX, 0.48, (0, 0, 0), 1, cv2.LINE_AA)
    return np.vstack([bar, resized])


def stack_rows(rows: list[np.ndarray]) -> np.ndarray:
    width = max(row.shape[1] for row in rows)
    padded = []
    for row in rows:
```

```
        if row.shape[1] == width:
            padded.append(row)
            continue
        canvas = np.full((row.shape[0], width, 3), 255, np.uint8)
        canvas[:, :row.shape[1]] = row
        padded.append(canvas)
    return np.vstack(padded)


def save_rgb(path: Path, image: np.ndarray) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    bgr = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
    cv2.imwrite(str(path), bgr)


def contact_sheet(records: list[tuple[str, list[tuple[str, np.ndarray]]]]) -> np.ndarray:
    """records: (row title, [(panel title, image), ...])."""
    rows = []
    for row_title, panels in records:
        tiles = [fit_panel(image, title) for title, image in panels]
        # We put a narrow name tag on the left of the row.
        tag_h = tiles[0].shape[0]
        tag = np.full((tag_h, 150, 3), 245, np.uint8)
        cv2.putText(tag, row_title[:18], (6, tag_h // 2), cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 0, 0), 1, cv2.LINE_AA)
        body = np.hstack(tiles)
        if tag.shape[0] != body.shape[0]:
            tag = cv2.resize(tag, (tag.shape[1], body.shape[0]))
        rows.append(np.hstack([tag, body]))
    return stack_rows(rows)


def choose_roles(images: dict[str, np.ndarray]) -> dict[str, str]:
    """The examples we chose for the paper: Johanneskirche for lines, obama for a person, Sanfrancisco for the failure. We fall back if a name is missing."""
    preferred = {
        "lines": "Johanneskirche",
        "saliency": "obama",
        "failure": "Sanfrancisco",
        "height": "waterfall",
        "enlarge": "butterfly",
    }
    if all(name in images for name in preferred.values()):
        return preferred
    line_score = {}
    saliency_peak = {}
    edge_fraction = {}
    for name, image in images.items():
        lum = luminance(image)
        lines = line_protection(lum, "vertical")
        sal = protection_maps(image)["saliency"]
        edges = cv2.Canny((lum * 255).astype(np.uint8), 80, 160)
        line_score[name] = float((lines > 0.75).mean())
        saliency_peak[name] = float(sal.max() / (sal.mean() + 1e-6))
        edge_fraction[name] = float(np.count_nonzero(edges)) / edges.size
    names = list(images)
    return {
        "lines": max(line_score, key=line_score.get),
        "saliency": max(saliency_peak, key=saliency_peak.get),
        "failure": max(edge_fraction, key=edge_fraction.get),
        "height": names[0],
        "enlarge": names[min(1, len(names) - 1)],
    }


def write_summary(rows: list[dict], path: Path) -> None:
    groups: dict[tuple[str, str], list[dict]] = {}
    for row in rows:
        groups.setdefault((row["setting"], row["method"]), []).append(row)
    lines = ["setting,method,n,mean_saliency_removed,mean_line_crossings_per_100,mean_runtime_s,all_sizes_ok"]
    for key in sorted(groups):
        group = groups[key]
        sal_vals = [float(r["saliency_removed_fraction"]) for r in group if r["saliency_removed_fraction"] != ""]
        line_vals = [float(r["line_crossings_per_100"]) for r in group if r["line_crossings_per_100"] != ""]
        run_vals = [float(r["runtime_s"]) for r in group]
        sal = sum(sal_vals) / len(sal_vals) if sal_vals else float("nan")
        crossed = sum(line_vals) / len(line_vals) if line_vals else float("nan")
        runtime = sum(run_vals) / len(run_vals)
        sizes_ok = all(int(r["size_ok"]) == 1 for r in group)
        lines.append(
```

```
            f"{key[0]},{key[1]},{len(group)},{sal:.6f},{crossed:.4f},{runtime:.4f},{int(sizes_ok)}"
        )
    path.write_text("\n".join(lines) + "\n", encoding="utf-8")


def main() -> None:
    RESULT_DIR.mkdir(parents=True, exist_ok=True)
    paths = discover_images()
    images = {path.stem: limit_long_side(load_rgb(path), MAX_SIDE) for path in paths}
    roles = choose_roles(images)
    (RESULT_DIR / "roles.txt").write_text(
        "\n".join(f"{role}={name}" for role, name in roles.items()) + "\n",
        encoding="utf-8",
    )
    print("roles", roles, flush=True)

    rows: list[dict] = []
    outputs: dict[tuple[str, str, str], np.ndarray] = {}

    width_75_methods = ["scale", "crop", "classic", "forward", "forward_saliency", "forward_lines", "ours"]
    width_50_methods = ["scale", "crop", "classic", "ours"]

    for name, image in images.items():
        height, width = image.shape[:2]
        jobs = [("width_75", (height, max(2, int(round(width * 0.75)))), width_75_methods)]
        jobs.append(("width_50", (height, max(2, int(round(width * 0.50)))), width_50_methods))
        if name == roles["height"]:
            jobs.append(("height_75", (max(2, int(round(height * 0.75))), width), ["scale", "classic", "ours"]))
        if name == roles["enlarge"]:
            jobs.append(("enlarge_150", (height, max(2, int(round(width * 1.50)))), ["scale", "ours"]))
        for setting, target, methods in jobs:
            for method in methods:
                print(f"{name} {setting} {method}", flush=True)
                output, info, elapsed = run_one(image, target, method)
                rows.append(row_for(name, setting, method, image, output, info, elapsed, target))
                outputs[(name, setting, method)] = output
                print(f"  {elapsed:.1f}s -> {output.shape[0]}x{output.shape[1]}", flush=True)

    csv_path = RESULT_DIR / "metrics.csv"
    with csv_path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)
    write_summary(rows, RESULT_DIR / "summary.csv")

    compare_75 = []
    compare_50 = []
    for name in PAPER_FIGURES:
        if name not in images:
            continue
        image = images[name]
        compare_75.append((name, [
            ("Original", image),
            ("Scale", outputs[(name, "width_75", "scale")]),
            ("Crop", outputs[(name, "width_75", "crop")]),
            ("Classic", outputs[(name, "width_75", "classic")]),
            ("Ours", outputs[(name, "width_75", "ours")]),
        ]))
        compare_50.append((name, [
            ("Original", image),
            ("Scale", outputs[(name, "width_50", "scale")]),
            ("Crop", outputs[(name, "width_50", "crop")]),
            ("Classic", outputs[(name, "width_50", "classic")]),
            ("Ours", outputs[(name, "width_50", "ours")]),
        ]))
    save_rgb(RESULT_DIR / "fig_compare_75.png", contact_sheet(compare_75))
    save_rgb(RESULT_DIR / "fig_compare_50.png", contact_sheet(compare_50))
    paper_names = [roles["lines"], roles["saliency"], roles["failure"], roles["height"]]
    by_name_75 = {name: row for name, row in compare_75}
    by_name_50 = {name: row for name, row in compare_50}
    save_rgb(
        RESULT_DIR / "fig_compare_75_paper.png",
        contact_sheet([(name, by_name_75[name]) for name in paper_names]),
    )
    save_rgb(
        RESULT_DIR / "fig_compare_50_paper.png",
        contact_sheet([(name, by_name_50[name]) for name in paper_names]),
    )

```

```
    height_name = roles["height"]
    height_image = images[height_name]
    save_rgb(RESULT_DIR / "fig_height.png", contact_sheet([(height_name, [
        ("Original", height_image),
        ("Scale", outputs[(height_name, "height_75", "scale")]),
        ("Classic", outputs[(height_name, "height_75", "classic")]),
        ("Ours", outputs[(height_name, "height_75", "ours")]),
    ])]))

    enlarge_name = roles["enlarge"]
    enlarge_image = images[enlarge_name]
    save_rgb(RESULT_DIR / "fig_enlarge.png", contact_sheet([(enlarge_name, [
        ("Original", enlarge_image),
        ("Scale", outputs[(enlarge_name, "enlarge_150", "scale")]),
        ("Ours", outputs[(enlarge_name, "enlarge_150", "ours")]),
    ])]))

    failure_name = roles["failure"]
    save_rgb(RESULT_DIR / "fig_failure.png", contact_sheet([(failure_name + " 50%", [
        ("Original", images[failure_name]),
        ("Scale", outputs[(failure_name, "width_50", "scale")]),
        ("Crop", outputs[(failure_name, "width_50", "crop")]),
        ("Classic", outputs[(failure_name, "width_50", "classic")]),
        ("Ours", outputs[(failure_name, "width_50", "ours")]),
    ])]))

    line_name = roles["lines"]
    line_image = images[line_name]
    save_rgb(RESULT_DIR / "fig_ablation.png", contact_sheet([(line_name, [
        ("Forward", outputs[(line_name, "width_75", "forward")]),
        ("+Saliency", outputs[(line_name, "width_75", "forward_saliency")]),
        ("+Lines", outputs[(line_name, "width_75", "forward_lines")]),
        ("Full", outputs[(line_name, "width_75", "ours")]),
        ("Classic", outputs[(line_name, "width_75", "classic")]),
    ])]))

    maps = protection_maps(line_image)
    seam = next_vertical_seam(line_image, method="ours", weights=WEIGHTS["ours"])
    save_rgb(RESULT_DIR / "fig_maps.png", contact_sheet([(line_name, [
        ("Input", line_image),
        ("Saliency", colorize(maps["saliency"])),
        ("Lines", colorize(maps["lines_vertical"])),
        ("Next seam", draw_vertical_seam(line_image, seam)),
    ])]))
    print("wrote", RESULT_DIR, flush=True)


if __name__ == "__main__":
    main()
```

Listing 10. experiments/run_experiments.py

```
"""At 50% width we compare a full carve with seams stopped at 75% and then scaled. A 75% target has to match the full model, or we do not trust the run."""

from __future__ import annotations

import csv
import sys
import time
from pathlib import Path

import numpy as np

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
sys.path.insert(0, str(ROOT / "experiments"))

from run_experiments import (  # noqa: E402
    RESULT_DIR,
    contact_sheet,
    discover_images,
    load_rgb,
    save_rgb,
)
from src.baselines import limit_long_side  # noqa: E402
from src.metrics import size_matches  # noqa: E402
from src.retarget import retarget  # noqa: E402

```

```
ORDER = [
    "brick_house",
    "butterfly",
    "child",
    "eagle",
    "Johanneskirche",
    "obama",
    "Sanfrancisco",
    "waterfall",
]
PAPER = ["Johanneskirche", "obama", "Sanfrancisco", "waterfall"]


def main() -> None:
    images = {
        path.stem: limit_long_side(load_rgb(path), 640) for path in discover_images()
    }
    check = images["Sanfrancisco"]
    height, width = check.shape[:2]
    target_75 = (height, max(2, int(round(width * 0.75))))
    ours_75, info_75 = retarget(check, target_75, method="ours", return_info=True)
    hybrid_75, info_h = retarget(check, target_75, method="hybrid", return_info=True)
    diff = int(np.abs(ours_75.astype(np.int16) - hybrid_75.astype(np.int16)).max())
    print(
        f"75% Sanfrancisco max abs diff {diff} "
        f"scaled {info_h.get('scaled_after_seams')} seams {info_75.get('vertical_seams')}",
        flush=True,
    )
    if diff != 0 or info_h.get("scaled_after_seams"):
        raise SystemExit("hybrid at 75% should match ours and should not scale")

    rows = []
    outputs = {}
    for name in ORDER:
        image = images[name]
        height, width = image.shape[:2]
        target = (height, max(2, int(round(width * 0.50))))
        outputs[(name, "original")] = image
        for method in ("scale", "ours", "hybrid"):
            print(f"{name} {method}", flush=True)
            started = time.perf_counter()
            output, info = retarget(image, target, method=method, return_info=True)
            elapsed = time.perf_counter() - started
            print(
                f"  {elapsed:.1f}s {output.shape[0]}x{output.shape[1]} "
                f"seams={info.get('vertical_seams')} scaled={info.get('scaled_after_seams')}",
                flush=True,
            )
            if not size_matches(output, target):
                raise SystemExit(f"size mismatch {name} {method}")
            outputs[(name, method)] = output
            rows.append({
                "image": name,
                "method": method,
                "target_w": target[1],
                "out_w": output.shape[1],
                "vertical_seams": info.get("vertical_seams", 0),
                "scaled_after_seams": info.get("scaled_after_seams", 0),
                "runtime_s": f"{elapsed:.4f}",
            })

    RESULT_DIR.mkdir(parents=True, exist_ok=True)
    csv_path = RESULT_DIR / "hybrid_50.csv"
    with csv_path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)

    def sheet(names: list[str]):
        records = []
        for name in names:
            records.append((name, [
                ("Original", outputs[(name, "original")]),
                ("Scale", outputs[(name, "scale")]),
                ("Ours", outputs[(name, "ours")]),
                ("Hybrid", outputs[(name, "hybrid")]),
            ]))
        return contact_sheet(records)

```

```
    save_rgb(RESULT_DIR / "fig_hybrid_50.png", sheet(PAPER))
    save_rgb(RESULT_DIR / "fig_hybrid_50_all.png", sheet(ORDER))
    print("wrote figures", flush=True)


if __name__ == "__main__":
    main()
```

Listing 11. experiments/run_hybrid.py

```
"""Print the write-up numbers from the saved CSVs. We do not carve here. We use the sample standard deviation with n-1. We chose not to score the hybrid."""

from __future__ import annotations

import csv
import statistics
from math import comb
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
CSV_PATH = ROOT / "experiments" / "results" / "metrics.csv"
N30_PATH = ROOT / "experiments" / "results" / "metrics_n30.csv"

# We chose w_s and w_l on these two images. The check drops both.
TUNING = ("Johanneskirche", "obama")

WIDTH_75 = (
    ("classic", "Classic"),
    ("forward", "Forward only"),
    ("forward_saliency", "Forward + saliency"),
    ("forward_lines", "Forward + lines"),
    ("ours", "Full model"),
)
WIDTH_50 = (
    ("classic", "Classic"),
    ("ours", "Full model"),
)
PER_IMAGE = (
    ("classic", "Classic"),
    ("forward", "Forward"),
    ("forward_saliency", "+Saliency"),
    ("forward_lines", "+Lines"),
    ("ours", "Full"),
)


def load_rows() -> list[dict[str, str]]:
    with CSV_PATH.open(newline="", encoding="utf-8") as handle:
        return list(csv.DictReader(handle))


def select(rows, setting: str, method: str, skip: tuple[str, ...] = ()) -> list[dict[str, str]]:
    chosen = []
    for row in rows:
        if row["setting"] != setting or row["method"] != method:
            continue
        if row["image"] in skip:
            continue
        chosen.append(row)
    return chosen


def mean_std(values: list[float]) -> tuple[float, float]:
    return statistics.fmean(values), statistics.stdev(values)


def print_summary(rows, setting: str, methods, skip: tuple[str, ...] = ()) -> None:
    label = setting if not skip else f"{setting} without {', '.join(skip)}"
    print(f"\n{label}")
    print(f"{'method':<22} {'n':>3} {'saliency':>16} {'lines':>16} {'seconds':>16}")
    for method, title in methods:
        group = select(rows, setting, method, skip)
        sal = [float(row["saliency_removed_fraction"]) for row in group]
        lines = [float(row["line_crossings_per_100"]) for row in group]
        seconds = [float(row["runtime_s"]) for row in group]
        sal_mean, sal_std = mean_std(sal)
        line_mean, line_std = mean_std(lines)
        sec_mean, sec_std = mean_std(seconds)
```

```
        print(
            f"{title:<22} {len(group):>3} "
            f"{sal_mean:7.3f} ({sal_std:5.3f}) "
            f"{line_mean:7.2f} ({line_std:5.2f}) "
            f"{sec_mean:7.1f} ({sec_std:4.1f})"
        )


def print_per_image(rows) -> None:
    order = []
    for row in rows:
        if row["setting"] == "width_75" and row["image"] not in order:
            order.append(row["image"])
    print("\nper-image width_75, saliency / lines")
    header = f"{'image':<16}" + "".join(f"{title:>16}" for _, title in PER_IMAGE)
    print(header)
    for image in order:
        cells = []
        mark = "*" if image in TUNING else ""
        for method, _title in PER_IMAGE:
            match = [
                row for row in rows
                if row["image"] == image and row["setting"] == "width_75" and row["method"] == method
            ]
            if len(match) != 1:
                raise SystemExit(f"expected one row for {image} {method}, found {len(match)}")
            row = match[0]
            sal = float(row["saliency_removed_fraction"])
            lines = float(row["line_crossings_per_100"])
            cells.append(f"{sal:.3f}/{lines:.2f}")
        body = "".join(f"{cell:>16}" for cell in cells)
        print(f"{image + mark:<16}{body}")
    print("* tuning image, dropped in the six-image check")


def print_lsd(rows) -> None:
    """LSD crossings per 100 seam pixels. We kept this map out of the cost."""
    if not rows or "lsd_crossings_per_100" not in rows[0]:
        print("\nlsd column is not in the csv")
        return
    print("\nlsd crossings per 100 seam pixels (not used to choose seams)")
    for setting in ("width_75", "width_50"):
        print(f"\n{setting}")
        print(f"{'method':<22} {'n':>3} {'lsd':>16}")
        for method, title in PER_IMAGE:
            group = select(rows, setting, method)
            values = [float(row["lsd_crossings_per_100"]) for row in group if row["lsd_crossings_per_100"] != ""]
            if len(values) < 2:
                print(f"{title:<22} {len(values):>3}  (not enough rows)")
                continue
            mean, std = mean_std(values)
            print(f"{title:<22} {len(values):>3} {mean:7.2f} ({std:5.2f})")
        order = []
        for row in rows:
            if row["setting"] == setting and row["image"] not in order and row["method"] == "classic":
                order.append(row["image"])
        header = f"{'image':<16}" + "".join(f"{title:>12}" for _, title in PER_IMAGE)
        print(header)
        for image in order:
            cells = []
            classic = full = None
            for method, _title in PER_IMAGE:
                match = [
                    row for row in rows
                    if row["image"] == image and row["setting"] == setting and row["method"] == method
                ]
                if len(match) != 1 or match[0]["lsd_crossings_per_100"] == "":
                    raise SystemExit(f"missing lsd score for {image} {setting} {method}")
                value = float(match[0]["lsd_crossings_per_100"])
                cells.append(f"{value:.2f}")
                if method == "classic":
                    classic = value
                if method == "ours":
                    full = value
            body = "".join(f"{cell:>12}" for cell in cells)
            relation = "full < classic" if full < classic else "full >= classic"
            print(f"{image:<16}{body}   {relation}")


```

```
def binomial_two_sided(successes: int, trials: int) -> float:
    """Two-sided exact binomial test at p = 0.5."""
    if trials <= 0:
        return float("nan")
    probs = [comb(trials, k) / 2**trials for k in range(trials + 1)]
    observed = probs[successes]
    return sum(prob for prob in probs if prob <= observed + 1e-15)


def print_width50_per_image(rows) -> None:
    """Classic and full at 50% width: saliency / Hough / LSD."""
    order = []
    for row in rows:
        if row["setting"] == "width_50" and row["method"] == "classic" and row["image"] not in order:
            order.append(row["image"])
    print("\nper-image width_50, classic and full, saliency/hough/lsd")
    print(f"{'image':<16}{'classic':>20}{'full':>20}  saliency  hough")
    sal_lower = hough_lower = lsd_lower = 0
    for image in order:
        cells = []
        values = {}
        for method in ("classic", "ours"):
            match = [
                row for row in rows
                if row["image"] == image and row["setting"] == "width_50" and row["method"] == method
            ]
            if len(match) != 1:
                raise SystemExit(f"expected one width_50 row for {image} {method}, found {len(match)}")
            row = match[0]
            sal = float(row["saliency_removed_fraction"])
            hough = float(row["line_crossings_per_100"])
            lsd = float(row["lsd_crossings_per_100"])
            values[method] = (sal, hough, lsd)
            cells.append(f"{sal:.3f}/{hough:.2f}/{lsd:.2f}")
        if values["ours"][0] < values["classic"][0]:
            sal_lower += 1
        if values["ours"][1] < values["classic"][1]:
            hough_lower += 1
        if values["ours"][2] < values["classic"][2]:
            lsd_lower += 1
        sal_rel = "full < classic" if values["ours"][0] < values["classic"][0] else "full >= classic"
        hough_rel = "full < classic" if values["ours"][1] < values["classic"][1] else "full >= classic"
        body = "".join(f"{cell:>20}" for cell in cells)
        print(f"{image:<16}{body}  {sal_rel:<16} {hough_rel}")
    print(
        f"full lower than classic on saliency {sal_lower}/{len(order)}, "
        f"hough {hough_lower}/{len(order)}, lsd {lsd_lower}/{len(order)}"
    )


def print_n30_saliency() -> None:
    if not N30_PATH.exists():
        print("\nmetrics_n30.csv is missing")
        return
    with N30_PATH.open(newline="", encoding="utf-8") as handle:
        rows = list(csv.DictReader(handle))
    order = []
    for row in rows:
        if row["image"] not in order:
            order.append(row["image"])
    headline = [name for name in order if name not in TUNING]
    by = {(row["image"], row["method"]): row for row in rows}
    print(f"\nn30 width_75 saliency, n={len(headline)}, seed sample in metrics_n30.csv")
    print(f"{'method':<22} {'n':>3} {'saliency':>16}")
    for method, title in PER_IMAGE:
        values = [float(by[(name, method)]["saliency_removed_fraction"]) for name in headline]
        mean, std = mean_std(values)
        print(f"{title:<22} {len(values):>3} {mean:7.3f} ({std:5.3f})")
    lower = higher = tied = 0
    for name in headline:
        classic = float(by[(name, "classic")]["saliency_removed_fraction"])
        full = float(by[(name, "ours")]["saliency_removed_fraction"])
        if full < classic:
            lower += 1
        elif full > classic:
            higher += 1
        else:
            tied += 1
    trials = lower + higher
```

```
    print(
        f"saliency sign test, full versus classic, lower/higher/tied "
        f"{lower}/{higher}/{tied}, "
        f"two-sided binomial p={binomial_two_sided(lower, trials):.4e} "
        f"on {trials} non-ties"
    )


LEARNED_METHODS = (
    ("classic", "Classic"),
    ("forward", "Forward only"),
    ("forward_lines", "Forward + lines"),
    ("ours_spectral", "Ours spectral"),
    ("forward_saliency_learned", "Learned saliency only"),
    ("ours_learned", "Ours learned"),
)
LEARNED_COLUMNS = (
    ("spectral_mass", "Spectral", 3),
    ("learned_mass", "Learned", 3),
    ("hough", "Hough", 2),
    ("lsd", "LSD", 2),
    ("learned_blob", "Blob", 3),
)


def format_p(p: float) -> str:
    if p < 1e-3:
        return f"{p:.4e}"
    return f"{p:.4f}"


def _learned_rows(path: Path) -> list[dict[str, str]]:
    with path.open(newline="", encoding="utf-8") as handle:
        return list(csv.DictReader(handle))


def analyze_learned(rows: list[dict[str, str]], setting: str, skip: tuple[str, ...] = ()) -> tuple[str, dict]:
    """The mean table and the sign tests. Lower is better. We wrote this text so it could go into the paper."""
    names = []
    for row in rows:
        if row["setting"] == setting and row["image"] not in names and row["image"] not in skip:
            names.append(row["image"])
    by = {(row["image"], row["setting"], row["method"]): row for row in rows}
    lines = [f"\nlearned cross-eval {setting} n={len(names)} skip={','.join(skip) if skip else 'none'}"]
    lines.append("ours_learned is compared with ours spectral and forward + lines. Not with forward-only, because that confounds w_l.")
    header = f"{'method':<24} {'n':>3}" + "".join(f"{title:>16}" for _key, title, _d in LEARNED_COLUMNS)
    lines.append(header)
    cells = {}
    for method, title in LEARNED_METHODS:
        group = []
        for name in names:
            row = by.get((name, setting, method))
            if row is None:
                raise SystemExit(f"missing {name} {setting} {method}")
            group.append(row)
        rendered = []
        for key, _title, digits in LEARNED_COLUMNS:
            values = [float(row[key]) for row in group]
            text = f"{statistics.fmean(values):.{digits}f} ({statistics.stdev(values):.{digits}f})"
            rendered.append(text)
            cells[(method, key)] = text
        lines.append(f"{title:<24} {len(group):>3}" + "".join(f"{item:>16}" for item in rendered))
    tests = {}
    for left, right, right_name in (
        ("ours_learned", "ours_spectral", "ours spectral"),
        ("ours_learned", "forward_lines", "forward + lines"),
    ):
        for key, map_name, _digits in (("spectral_mass", "spectral", 3), ("learned_mass", "learned", 3)):
            lower = higher = tied = 0
            for name in names:
                a = float(by[(name, setting, left)][key])
                b = float(by[(name, setting, right)][key])
                if a < b:
                    lower += 1
                elif a > b:
                    higher += 1
                else:
                    tied += 1
            trials = lower + higher
```

```
            p_text = format_p(binomial_two_sided(lower, trials))
            line = (
                f"sign {left} vs {right} on {map_name}: lower/higher/tied "
                f"{lower}/{higher}/{tied} p={p_text} on {trials} non-ties"
            )
            lines.append(line)
            tests[(right, key)] = {
                "lower": lower,
                "higher": higher,
                "tied": tied,
                "p": p_text,
                "right_name": right_name,
                "map_name": map_name,
            }
    matrix = ["2x2 mean spectral_mass / learned_mass (chosen row, scored column)"]
    for method, title in (("ours_spectral", "chosen spectral"), ("ours_learned", "chosen learned")):
        matrix.append(f"{title}: {cells[(method, 'spectral_mass')]} / {cells[(method, 'learned_mass')]}")
    lines.extend(matrix)
    payload = {"n": len(names), "names": names, "cells": cells, "tests": tests, "text": "\n".join(lines) + "\n"}
    return payload["text"], payload


def format_learned_report() -> str:
    parts = []
    eight = ROOT / "experiments" / "results" / "metrics_learned_8.csv"
    thirty = ROOT / "experiments" / "results" / "metrics_learned_n30.csv"
    if eight.exists():
        rows = _learned_rows(eight)
        for setting in ("width_75", "width_50"):
            text, _payload = analyze_learned(rows, setting)
            parts.append(text)
    else:
        parts.append("\nmetrics_learned_8.csv is missing\n")
    if thirty.exists():
        rows = _learned_rows(thirty)
        present = {row["image"] for row in rows}
        skipped = tuple(name for name in TUNING if name in present)
        text, _payload = analyze_learned(rows, "width_75", skip=skipped)
        parts.append(text)
        parts.append(
            "tuning images in the 30-image file: "
            + (", ".join(skipped) if skipped else "none")
            + "\n"
        )
    else:
        parts.append("\nmetrics_learned_n30.csv is missing\n")
    return "".join(parts)


def main() -> None:
    rows = load_rows()
    print(f"source {CSV_PATH}")
    print("sample standard deviation (n - 1)")
    print_summary(rows, "width_75", WIDTH_75)
    print_summary(rows, "width_50", WIDTH_50)
    print_per_image(rows)
    print_width50_per_image(rows)
    print("\nsix-image check, tuning images dropped")
    print_summary(rows, "width_75", WIDTH_75, skip=TUNING)
    print_lsd(rows)
    print_n30_saliency()


if __name__ == "__main__":
    main()
```

Listing 12. experiments/report_numbers.py

```
"""We add an LSD score to metrics.csv. We do not change the cost. If the saved saliency or Hough scores do not match, we stop."""

from __future__ import annotations

import csv
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
sys.path.insert(0, str(ROOT / "experiments"))
```

```
12
13  from run_experiments import (  # noqa: E402
14      RESULT_DIR,
15      WEIGHTS,
16      discover_images,
17      load_rgb,
18      run_one,
19  )
20  from src.baselines import limit_long_side  # noqa: E402
21  from src.metrics import (  # noqa: E402
22      line_crossings_per_100,
23      lsd_crossings_per_100,
24      saliency_removed_fraction,
25  )
26
27  CSV_PATH = RESULT_DIR / "metrics.csv"
28  METHODS = ("classic", "forward", "forward_saliency", "forward_lines", "ours")
29  COLUMN = "lsd_crossings_per_100"
30
31
32  def targets(image):
33      height, width = image.shape[:2]
34      return {
35          "width_75": (height, max(2, int(round(width * 0.75)))),
36          "width_50": (height, max(2, int(round(width * 0.50)))),
37      }
38
39
40  def main() -> None:
41      with CSV_PATH.open(newline="", encoding="utf-8") as handle:
42          reader = csv.DictReader(handle)
43          fieldnames = list(reader.fieldnames or [])
44          rows = list(reader)
45      if COLUMN not in fieldnames:
46          fieldnames.append(COLUMN)
47      by_key = {(row["image"], row["setting"], row["method"]): row for row in rows}
48
49      images = {
50          path.stem: limit_long_side(load_rgb(path), 640) for path in discover_images()
51      }
52      fresh: dict[tuple[str, str, str], str] = {}
53      new_rows: list[dict[str, str]] = []
54
55      for name in list(dict.fromkeys(row["image"] for row in rows)):
56          image = images[name]
57          for setting, target in targets(image).items():
58              saved = by_key.get((name, setting, "classic"))
59              if saved is not None:
60                  if (int(saved["target_h"]), int(saved["target_w"])) != target:
61                      raise SystemExit(f"target mismatch {name} {setting}: csv {saved['target_h']}x{saved['target_w']} vs {target}")
62              for method in METHODS:
63                  print(f"{name} {setting} {method}", flush=True)
64                  output, info, elapsed = run_one(image, target, method)
65                  sal = f"{saliency_removed_fraction(info):.6f}"
66                  lines = f"{line_crossings_per_100(info):.4f}"
67                  lsd = f"{lsd_crossings_per_100(info):.4f}"
68                  print(f"  sal {sal} hough {lines} lsd {lsd} {elapsed:.1f}s", flush=True)
69                  key = (name, setting, method)
70                  existing = by_key.get(key)
71                  if existing is not None:
72                      if existing["saliency_removed_fraction"] != sal or existing["line_crossings_per_100"] != lines:
73                          raise SystemExit(
74                              f"seam score changed for {name} {setting} {method}: "
75                              f"csv {existing['saliency_removed_fraction']}/{existing['line_crossings_per_100']} "
76                              f"rerun {sal}/{lines}"
77                          )
78                  else:
79                      new_rows.append({
80                          "image": name,
81                          "setting": setting,
82                          "method": method,
83                          "w_s": str(WEIGHTS[method][0]),
84                          "w_l": str(WEIGHTS[method][1]),
85                          "target_h": str(target[0]),
86                          "target_w": str(target[1]),
87                          "out_h": str(output.shape[0]),
88                          "out_w": str(output.shape[1]),
89                          "size_ok": "1" if output.shape[:2] == target else "0",
90                          "saliency_removed_fraction": sal,
```

```
                        "line_crossings_per_100": lines,
                        "runtime_s": f"{elapsed:.4f}",
                        COLUMN: lsd,
                    })
                fresh[key] = lsd

    for row in rows:
        key = (row["image"], row["setting"], row["method"])
        row[COLUMN] = fresh.get(key, row.get(COLUMN, ""))
    rows.extend(new_rows)

    with CSV_PATH.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=fieldnames, extrasaction="ignore", lineterminator="\n")
        writer.writeheader()
        writer.writerows(rows)
    print(f"wrote {COLUMN} for {len(fresh)} runs, appended {len(new_rows)} rows", flush=True)


if __name__ == "__main__":
    main()
```

Listing 13. experiments/score_lsd.py

```
"""A seed-604 sample of 30 images at 75% width. We score LSD in the same carve. We trust a new image only after the original eight still match metrics.csv."""

from __future__ import annotations

import csv
import random
import sys
import urllib.request
import zipfile
from math import comb
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
sys.path.insert(0, str(ROOT / "experiments"))

from run_experiments import RESULT_DIR, load_rgb, run_one  # noqa: E402
from src.baselines import limit_long_side  # noqa: E402

SEED = 604
SAMPLE_N = 30
ZIP_URL = "https://people.csail.mit.edu/mrub/retargetme/download/images-20100824.zip"
ZIP_PATH = ROOT / "experiments" / "retargetme_source.zip"
ORIGINAL_DIR = ROOT / "experiments" / "retargetme"
SAMPLE_DIR = ROOT / "experiments" / "retargetme_n30"
OLD_CSV = RESULT_DIR / "metrics.csv"
OUT_CSV = RESULT_DIR / "metrics_n30.csv"
SUMMARY_PATH = RESULT_DIR / "n30_summary.txt"

ORIGINAL = (
    "brick_house",
    "butterfly",
    "child",
    "eagle",
    "Johanneskirche",
    "obama",
    "Sanfrancisco",
    "waterfall",
)
TUNING = ("Johanneskirche", "obama")
METHODS = ("classic", "forward", "forward_saliency", "forward_lines", "ours")
SETTING = "width_75"
ANCHOR = ("brick_house", "classic", "0.153154", "3.2319")


def download_zip() -> None:
    if ZIP_PATH.exists() and ZIP_PATH.stat().st_size > 0:
        return
    print(f"downloading {ZIP_URL}", flush=True)
    request = urllib.request.Request(ZIP_URL, headers={"User-Agent": "ee604-retarget/1.0"})
    with urllib.request.urlopen(request, timeout=120) as response:
        ZIP_PATH.write_bytes(response.read())
    print(f"zip bytes {ZIP_PATH.stat().st_size}", flush=True)


```

```
def archive_pngs() -> dict[str, str]:
    download_zip()
    with zipfile.ZipFile(ZIP_PATH) as archive:
        names = {}
        for member in archive.namelist():
            path = Path(member)
            if path.suffix.lower() == ".png":
                names[path.stem] = member
    if len(names) < SAMPLE_N:
        raise SystemExit(f"archive has {len(names)} pngs, need at least {SAMPLE_N}")
    return names


def choose_sample(stems: list[str]) -> list[str]:
    rng = random.Random(SEED)
    sample = sorted(rng.sample(sorted(stems), SAMPLE_N))
    return sample


def extract_sample(members: dict[str, str], sample: list[str]) -> None:
    SAMPLE_DIR.mkdir(parents=True, exist_ok=True)
    with zipfile.ZipFile(ZIP_PATH) as archive:
        for stem in sample:
            dest = SAMPLE_DIR / f"{stem}.png"
            if dest.exists():
                continue
            dest.write_bytes(archive.read(members[stem]))
            print("extracted", stem, flush=True)


def load_working(stem: str):
    original = ORIGINAL_DIR / f"{stem}.png"
    path = original if original.exists() else SAMPLE_DIR / f"{stem}.png"
    return limit_long_side(load_rgb(path), 640)


def old_rows() -> dict[tuple[str, str, str], dict[str, str]]:
    with OLD_CSV.open(newline="", encoding="utf-8") as handle:
        rows = list(csv.DictReader(handle))
    return {(row["image"], row["setting"], row["method"]): row for row in rows}


def check_saved(saved: dict, name: str, method: str, info) -> None:
    from src.metrics import line_crossings_per_100, saliency_removed_fraction

    row = saved[(name, SETTING, method)]
    sal = f"{saliency_removed_fraction(info):.6f}"
    lines = f"{line_crossings_per_100(info):.4f}"
    if row["saliency_removed_fraction"] != sal or row["line_crossings_per_100"] != lines:
        raise SystemExit(
            f"regression failed {name} {method}: "
            f"csv {row['saliency_removed_fraction']}/{row['line_crossings_per_100']} "
            f"rerun {sal}/{lines}"
        )


def carve(name: str, saved_info: dict | None = None):
    image = load_working(name)
    height, width = image.shape[:2]
    target = (height, max(2, int(round(width * 0.75))))
    rows = []
    for method in METHODS:
        print(f"{name} {SETTING} {method}", flush=True)
        output, info, elapsed = run_one(image, target, method)
        if saved_info is not None:
            check_saved(saved_info, name, method, info)
        from run_experiments import row_for

        row = row_for(name, SETTING, method, image, output, info, elapsed, target)
        print(
            f"  sal {row['saliency_removed_fraction']} hough {row['line_crossings_per_100']} "
            f"lsd {row['lsd_crossings_per_100']} {elapsed:.1f}s",
            flush=True,
        )
        rows.append(row)
    return rows


def binomial_two_sided(successes: int, trials: int) -> float:
```

```
    """Two-sided exact binomial test at p = 0.5."""
    if trials <= 0:
        return float("nan")
    probs = [comb(trials, k) / 2**trials for k in range(trials + 1)]
    observed = probs[successes]
    return sum(prob for prob in probs if prob <= observed + 1e-15)


def summarize(rows: list[dict], sample: list[str]) -> str:
    import statistics

    headline = [name for name in sample if name not in TUNING]
    lines = [
        f"seed {SEED}",
        f"sample {len(sample)}: {', '.join(sample)}",
        f"headline drops {', '.join(TUNING)}; n={len(headline)}",
        "setting width_75, sample standard deviation (n - 1)",
        "LSD is not inside the cost",
        "",
    ]
    by = {(row["image"], row["method"]): row for row in rows}

    def values(method: str, names: list[str], column: str) -> list[float]:
        return [float(by[(name, method)][column]) for name in names]

    lines.append(f"{'method':<22} {'n':>3} {'saliency':>16} {'hough':>16} {'lsd':>16}")
    for method in METHODS:
        sal = values(method, headline, "saliency_removed_fraction")
        hough = values(method, headline, "line_crossings_per_100")
        lsd = values(method, headline, "lsd_crossings_per_100")
        lines.append(
            f"{method:<22} {len(headline):>3} "
            f"{statistics.fmean(sal):7.3f} ({statistics.stdev(sal):5.3f}) "
            f"{statistics.fmean(hough):7.2f} ({statistics.stdev(hough):5.2f}) "
            f"{statistics.fmean(lsd):7.2f} ({statistics.stdev(lsd):5.2f})"
        )

    def signs(left: str, right: str, column: str) -> tuple[int, int, int]:
        lower = higher = tied = 0
        for name in headline:
            a = float(by[(name, left)][column])
            b = float(by[(name, right)][column])
            if b < a:
                lower += 1
            elif b > a:
                higher += 1
            else:
                tied += 1
        return lower, higher, tied

    full_lower, full_higher, full_tied = signs("classic", "ours", "lsd_crossings_per_100")
    trials = full_lower + full_higher
    lines.append("")
    lines.append(
        f"LSD sign test, full versus classic, lower/higher/tied "
        f"{full_lower}/{full_higher}/{full_tied}, "
        f"two-sided binomial p={binomial_two_sided(full_lower, trials):.4f} "
        f"on {trials} non-ties"
    )
    line_lower, line_higher, line_tied = signs("forward", "forward_lines", "lsd_crossings_per_100")
    line_trials = line_lower + line_higher
    lines.append(
        f"LSD sign test, +lines versus forward-only, lower/higher/tied "
        f"{line_lower}/{line_higher}/{line_tied}, "
        f"two-sided binomial p={binomial_two_sided(line_lower, line_trials):.4f} "
        f"on {line_trials} non-ties"
    )
    sal_lower, sal_higher, sal_tied = signs("classic", "ours", "saliency_removed_fraction")
    sal_trials = sal_lower + sal_higher
    lines.append(
        f"saliency sign test, full versus classic, lower/higher/tied "
        f"{sal_lower}/{sal_higher}/{sal_tied}, "
        f"two-sided binomial p={binomial_two_sided(sal_lower, sal_trials):.4e} "
        f"on {sal_trials} non-ties"
    )
    text = "\n".join(lines) + "\n"
    return text


```

```
def write_csv(rows: list[dict]) -> None:
    RESULT_DIR.mkdir(parents=True, exist_ok=True)
    with OUT_CSV.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0].keys()), lineterminator="\n")
        writer.writeheader()
        writer.writerows(rows)


def print_learned_reports() -> None:
    """Cross-evaluation of the learned-saliency CSVs. We do not carve again here."""
    from report_numbers import format_learned_report

    print(format_learned_report(), flush=True)


def summary_from_csv() -> None:
    """We rewrite n30_summary.txt from metrics_n30.csv. We do not carve again here."""
    with OUT_CSV.open(newline="", encoding="utf-8") as handle:
        rows = list(csv.DictReader(handle))
    sample = []
    for row in rows:
        if row["image"] not in sample:
            sample.append(row["image"])
    text = summarize(rows, sample)
    SUMMARY_PATH.write_text(text, encoding="utf-8")
    print(text, flush=True)


def main() -> None:
    if len(sys.argv) > 1 and sys.argv[1] == "--summary-only":
        summary_from_csv()
        return
    if len(sys.argv) > 1 and sys.argv[1] == "--learned":
        print_learned_reports()
        return
    members = archive_pngs()
    sample = choose_sample(list(members))
    print("sample", sample, flush=True)
    extract_sample(members, sample)
    saved = old_rows()
    anchor = saved[("brick_house", SETTING, "classic")]
    if (anchor["saliency_removed_fraction"], anchor["line_crossings_per_100"]) != ANCHOR[2:]:
        raise SystemExit(f"metrics.csv anchor moved: {anchor['saliency_removed_fraction']} {anchor['line_crossings_per_100']}")

    reused: dict[str, list[dict]] = {}
    print("regression on the original eight", flush=True)
    for name in ORIGINAL:
        reused[name] = carve(name, saved)
    print("regression passed", flush=True)

    rows: list[dict] = []
    for name in sample:
        if name in reused:
            print(f"reuse regression rows for {name}", flush=True)
            rows.extend(reused[name])
        else:
            rows.extend(carve(name))
        write_csv(rows)

    text = summarize(rows, sample)
    SUMMARY_PATH.write_text(text, encoding="utf-8")
    print(text, flush=True)
    print(f"wrote {OUT_CSV}", flush=True)


if __name__ == "__main__":
    main()
```

Listing 14. experiments/run_n30.py